

FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA - UNIFOR
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO

COMPRESSÃO SEMÂNTICA NA GERAÇÃO DE MODELOS BPMN 2.0 POR MODELOS DE LINGUAGEM: DSL E TRANSPILAÇÃO DETERMINÍSTICA

RAFAEL MARINS FLORENZANO

FORTALEZA – CEARÁ
2026

RAFAEL MARINS FLORENZANO

COMPRESSÃO SEMÂNTICA NA GERAÇÃO DE MODELOS BPMN 2.0 POR MODELOS DE LINGUAGEM: DSL E TRANSPILAÇÃO DETERMINÍSTICA

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Centro de Ciências Tecnológicas da Universidade de Fortaleza, como requisito parcial à obtenção do grau de bacharel em Engenharia da Computação.

Orientador: Prof. Caio Marques

FORTALEZA – CEARÁ

2026

A ficha catalográfica deve ser gerada no site da biblioteca da Unifor através do link https://goo.gl/XYUWSC (link encurtado).

Preencha o formulário com as informações solicitadas e ao final será gerado um arquivo PDF da ficha catalográfica a ser anexada na versão final do TCC.

O arquivo PDF deve ser renomeado para "ficha-catalografica.pdf" (sem aspas) e colocado no diretório "elementos-pre-textuais" (sem aspas) do modelo de TCC da Unifor.

Ficha catalográfica da obra elaborada pelo autor através do programa de geração automática da Biblioteca Central da Universidade de Fortaleza

---

Batista, Bruno .
TEORIA DA RELATIVIDADE: SUBTÍTULO / Bruno Batista, Sandra Lima. - 2017
40 f.

Trabalho de Conclusão de Curso (Graduação) - Universidade de Fortaleza. Curso de Ciência da Computação, Fortaleza, 2017.
Orientação: Liadina Camargo.

1. FÍSICA. 2. RELATIVIDADE. 3. TEMPO. 4. ESPAÇO. I. Lima, Sandra. II. Camargo, Liadina. III. Título.

RAFAEL MARINS FLORENZANO

COMPRESSÃO SEMÂNTICA NA GERAÇÃO DE MODELOS BPMN 2.0 POR MODELOS DE LINGUAGEM: DSL E TRANSPILAÇÃO DETERMINÍSTICA

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Centro de Ciências Tecnológicas da Universidade de Fortaleza, como requisito parcial à obtenção do grau de bacharel em Engenharia da Computação.

Aprovada em:

BANCA EXAMINADORA

Prof. Caio Marques (Orientador)
Centro de Ciências Tecnológicas - CCT
Universidade de Fortaleza - UNIFOR

Membro da Banca Dois
Centro de Ciências e Tecnologia - CCT
Universidade do Membro da Banca Dois - SIGLA

Membro da Banca Três
Centro de Ciências e Tecnologia - CCT
Universidade do Membro da Banca Três - SIGLA

Membro da Banca Quatro
Centro de Ciências e Tecnologia - CCT
Universidade do Membro da Banca Quatro - SIGLA

RESUMO

Este trabalho investiga se a geração de uma representação intermediária comprimida, seguida de expansão determinística, produz modelos BPMN 2.0 válidos de forma mais confiável e mais econômica do que a geração direta de XML por modelos de linguagem. Projetou-se uma linguagem de domínio específico com gramática EBNF formalmente especificada e um transpilador determinístico que a expande em XML validado contra o esquema oficial da OMG. Construiu-se um corpus de 1.021 processos a partir de fontes públicas de descrições textuais e especializou-se um modelo de sete bilhões de parâmetros por ajuste supervisionado com adaptação de baixo posto sobre pesos quantizados. A avaliação seguiu protocolo definido antes da execução, com 53 processos de referência modelados por especialistas e três repetições por item. O componente determinístico produziu XML válido em 100% das amostras e preservou a equivalência topológica em 99,6%, e a representação proposta mostrou-se seis vezes mais compacta que o XML lógico equivalente. Na primeira versão da notação, descrita no *prompt*, modelos de fronteira (DeepSeek e GLM) produziram saída válida em 64,8% e 56,6% das gerações, respectivamente, contra 98,1% e 94,3% da geração direta de XML pelos mesmos modelos, contrariando a hipótese principal. A análise das falhas atribuiu a maior parte delas não à novidade da linguagem, mas a um traço de seu desenho: referenciar um elemento exigia marcá-lo no momento em que era escrito, antecipando ramos ainda não gerados. Acrescentada a referência pelo rótulo de elementos já escritos, em emenda posterior à observação dos resultados e de valor exploratório, o mesmo modelo atingiu 86,8% de validade, com fidelidade topológica estatisticamente indistinguível da geração direta de XML e custo por requisição 47% menor. O modelo pequeno especializado, sem a gramática no contexto, alcançou os mesmos 86,8%, patamar replicado em 83,0% sobre duas famílias adicionais de modelos, com fidelidade inferior à do modelo de fronteira e custo por requisição de duas a quatro vezes menor sob preço de mercado, após treinamento de menos de um dólar. Mediu-se ainda a concordância entre especialistas distintos para um mesmo processo, com F1 estrutural de 0,1449, patamar em que operam todos os métodos avaliados: métricas estruturais que identificam nós por rótulo saturam próximo ao ruído e não devem ser lidas como percentual de acerto. Conclui-se que notações destinadas a modelos autorregressivos devem permitir referência ao que já foi escrito e que, satisfeita essa condição, o ajuste fino de um modelo pequeno é uma troca de custo por fidelidade, e não condição de viabilidade da abordagem.

Palavras-chave: BPMN; linguagem de domínio específico; modelos de linguagem; ajuste fino supervisionado; compressão de *tokens*.

ABSTRACT

This work investigates whether generating a compressed intermediate representation, followed by deterministic expansion, yields valid BPMN 2.0 models more reliably and more economically than direct XML generation by language models. A domain-specific language with a formally specified EBNF grammar was designed, together with a deterministic transpiler that expands it into XML validated against the official OMG schema. A corpus of 1,021 processes was built from public sources of textual process descriptions, and a seven-billion-parameter model was specialized through supervised fine-tuning with low-rank adaptation over quantized weights. The evaluation followed a protocol defined before execution, with 53 reference processes modeled by experts and three repetitions per item. The deterministic component produced valid XML for 100% of the samples and preserved topological equivalence in 99.6%, and the proposed representation proved six times more compact than the equivalent logical XML. In the first version of the notation, described in the prompt, frontier models (DeepSeek and GLM) produced valid output in 64.8% and 56.6% of generations, respectively, against 98.1% and 94.3% for direct XML generation by the same models, contrary to the main hypothesis. Failure analysis traced most of these failures not to the novelty of the language but to a feature of its design: referencing an element required marking it at the moment it was written, anticipating branches not yet generated. Once references by the label of already-written elements were added, in an amendment made after the results were observed and of exploratory value, the same model reached 86.8% validity, with topological fidelity statistically indistinguishable from direct XML generation and a 47% lower cost per request. The specialized small model, without the grammar in context, reached the same 86.8%, a level replicated at 83.0% across two additional model families, with lower fidelity than the frontier model and a cost per request two to four times lower at market prices, after training that cost less than one dollar. Agreement between different experts modeling the same process was also measured, yielding a structural F1 of 0.1449, the range in which all evaluated methods operate: structural metrics that identify nodes by label saturate close to noise and should not be read as percentage of correctness. The conclusion is that notations meant for autoregressive models should allow references to what has already been written and that, once this condition holds, fine-tuning a small model is a trade of cost for fidelity rather than a condition for the viability of the approach.

Keywords: BPMN; domain-specific language; language models; supervised fine-tuning; token compression.

LISTA DE ILUSTRAÇÕES

Figura 1 – Visão geral: construção do corpus e avaliação 36
Figura 2 – Validade por braço experimental 61

LISTA DE TABELAS

Tabela 1 – Fontes de dados e uso no trabalho 36
Tabela 2 – Configuração do ajuste supervisionado 43
Tabela 3 – Braços experimentais 44
Tabela 4 – Preços usados no cálculo do custo (US$ por milhão de *tokens*) 47
Tabela 5 – Validação por perturbação controlada 50
Tabela 6 – Desvios do protocolo inicial 54
Tabela 7 – Panorama dos braços 60
Tabela 8 – Contrastes de validade 62
Tabela 9 – Causas de invalidade nos braços de DSL 63

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Processo com decisão e repetição na BPMN-DSL 40
Código-fonte 2 – XML gerado pelo transpilador a partir do Código 1 40

LISTA DE SÍMBOLOS

$\mathscr{R}$	Multiconjunto *direct-follows* do modelo de referência
$\mathscr{C}$	Multiconjunto *direct-follows* do modelo candidato
P	Precisão sobre o multiconjunto *direct-follows*
R	Revocação sobre o multiconjunto *direct-follows*
(a,b)	Par de atividades em que a é diretamente seguida por b
n	Número de itens do conjunto de avaliação
k	Número de repetições de geração por item
p	Valor-p do teste de Wilcoxon pareado
p_{Holm}	Valor-p após correção de Holm para múltiplas comparações
r_{rb}	Tamanho de efeito *rank-biserial*
r	Recompensa composta
r'	Recompensa restrita a componentes verificáveis sem referência
r_{sint}	Componente de validade sintática da recompensa
r_{sem}	Componente de adequação semântica da recompensa
r_{topo}	Componente de coerência topológica da recompensa
r_{comp}	Componente de economia de *tokens* da recompensa
α	Algoritmo α de descoberta de processos

SUMÁRIO

1 INTRODUÇÃO 15
1.1 MOTIVAÇÃO 18
1.2 OBJETIVOS 19
1.2.1 Objetivo Geral 19
1.2.2 Objetivos Específicos 19
1.3 ORGANIZAÇÃO DO TRABALHO 19
2 FUNDAMENTAÇÃO TEÓRICA 21
2.1 MODELAGEM DE PROCESSOS DE NEGÓCIO E A NOTAÇÃO BPMN 2.0 21
2.1.1 Gerenciamento de Processos de Negócio 21
2.1.2 A Notação BPMN 2.0 22
2.1.3 Representação XML e Verbosidade 22
2.2 MODELOS DE LINGUAGEM DE GRANDE ESCALA 23
2.2.1 Arquitetura Transformer 23
2.2.2 Tokenização e o Custo de Sequências Longas 23
2.2.3 Geração de Saídas Estruturadas 24
2.3 LINGUAGENS DE DOMÍNIO ESPECÍFICO E COMPRESSÃO SEMÂNTICA 24
2.3.1 Linguagens de Domínio Específico 24
2.3.2 Compressão Semântica como Estratégia de Geração 25
2.4 ADAPTAÇÃO DE MODELOS DE LINGUAGEM A TAREFAS ESPECÍFICAS 26
2.4.1 Supervised Fine-Tuning 26
2.4.2 Ajuste com Eficiência de Parâmetros: LoRA e QLoRA 26
2.4.3 Aprendizado por Reforço com GRPO 26
2.5 EXTRAÇÃO DE PROCESSOS A PARTIR DE TEXTO NATURAL 27
2.5.1 Caracterização do Problema 27
2.5.2 Recursos e Datasets Disponíveis 27
2.6 AVALIAÇÃO DE GERAÇÃO DE ARTEFATOS ESTRUTURADOS 28
2.6.1 Validação por Esquema 28
2.6.2 Métricas Sintáticas e Semânticas 28
2.6.3 Distância de Edição em Grafos 29
2.6.4 Eficiência: Razão de Compressão de Tokens 29
2.6.5 Função de Recompensa Composta 29

2.7 SÍNTESE DO CAPÍTULO 30
3 TRABALHOS RELACIONADOS 31
3.1 GERAÇÃO DE MODELOS DE PROCESSO COM LLMS 31
3.2 REPRESENTAÇÕES COMPACTAS DE PROCESSOS 31
3.3 COMPRESSÃO DE *TOKENS*: ENTRADA *VERSUS* SAÍDA 33
3.4 MODELOS PEQUENOS ESPECIALIZADOS 33
3.5 GARANTIA DE VALIDADE SINTÁTICA 34
4 METODOLOGIA 35
4.1 VISÃO GERAL 35
4.2 DADOS 36
4.2.1 Fontes 36
4.2.2 Geração dos pares de treino 37
4.2.3 Partições e prevenção de contaminação 38
4.3 A BPMN-DSL 39
4.4 TRANSPILAÇÃO 41
4.5 ESPECIALIZAÇÃO DO MODELO 42
4.6 BRAÇOS EXPERIMENTAIS 43
4.7 MEDIDAS 45
4.7.1 Validade 46
4.7.2 Custo por requisição 46
4.7.3 Razão de compressão 47
4.7.4 Fidelidade 48
4.7.4.1 O juiz 48
4.7.4.2 Validação contra notas de especialista 49
4.7.4.3 Validação por perturbação controlada 50
4.7.4.4 Uso da medida 51
4.7.5 A medida originalmente prevista: DF-F1 51
4.8 PROTOCOLO E ANÁLISE ESTATÍSTICA 52
4.8.1 Execução 52
4.8.2 Unidade de análise e comparações 52
4.8.3 Equivalência 53
4.8.4 Custo e compressão 53
4.8.5 Estatuto dos resultados 53

4.9 DESVIOS DO PROTOCOLO INICIAL 54
4.9.1 Mudanças nos artefatos 54
4.9.2 Mudanças nos dados 55
4.9.3 Mudanças nas medidas e na análise 56
4.10 AMEAÇAS À VALIDADE 56
4.10.1 Validade interna 57
4.10.2 Validade de construto 57
4.10.3 Validade externa 58
4.10.4 Validade da conclusão 58
5 RESULTADOS 60
5.1 VISÃO GERAL 60
5.2 VALIDADE 61
5.2.1 Validade por braço 61
5.2.2 Comparações com margem de equivalência 62
5.2.3 Causas de invalidade 62
5.3 CUSTO 63
5.4 COMPRESSÃO 63
5.5 FIDELIDADE (EXPLORATÓRIA) 63
5.6 O MODELO ESPECIALIZADO 63
5.7 SÍNTESE 63
6 CONCLUSÕES E TRABALHOS FUTUROS 64
6.1 CONTRIBUIÇÕES DO TRABALHO 64
6.2 LIMITAÇÕES 66
6.3 TRABALHOS FUTUROS 67
REFERÊNCIAS 69

APÊNDICES 72
APÊNDICE A – Prompts dos braços experimentais 73
APÊNDICE B – Prompts do juiz de fidelidade e da paráfrase 74
APÊNDICE C – Piloto de escolha do modelo gerador 75
APÊNDICE D – Resultados das hipóteses originais 76

1 INTRODUÇÃO

Organizações dependem de processos documentados para operar de forma previsível, treinar pessoas e cumprir exigências regulatórias. A notação BPMN 2.0, padronizada pela Object Management Group (2011), tornou-se a linguagem de referência para essa documentação, com uso consolidado tanto na indústria quanto na literatura de gestão de processos (WESKE, 2019; DUMAS *et al.*, 2018). Na prática, porém, a maior parte do conhecimento sobre processos permanece registrada em linguagem natural: manuais internos, descrições de procedimentos e documentos de política. A conversão desse material em modelos formais continua sendo uma tarefa manual, lenta e dependente de especialistas (FRIEDRICH; MENDLING; PUHLMANN, 2011; AA *et al.*, 2018).

O avanço dos modelos de linguagem de grande porte (VASWANI *et al.*, 2017; BROWN *et al.*, 2020) tornou plausível automatizar essa conversão. A dificuldade está na forma de saída. A serialização XML da BPMN foi concebida para intercâmbio entre ferramentas, não para ser escrita diretamente por pessoas ou por modelos. Ao solicitar que um modelo produza XML BPMN completo, transfere-se a ele uma responsabilidade pouco adequada ao seu funcionamento: reproduzir com precisão um grande volume de marcação técnica, ainda que boa parte dessa marcação possa ser derivada de maneira determinística. O custo aparece em duas frentes. A primeira é econômica e não depende de verificação: cada *token* de marcação repetitiva consome contexto, tempo e dinheiro, e a quantidade emitida é diretamente mensurável. A segunda é de confiabilidade, e tem natureza distinta, é uma **expectativa a ser testada**, não um dado. O risco existe em princípio, pois uma única inconsistência estrutural invalida o documento inteiro; se ele de fato se materializa, e em que medida, é questão empírica. Adianta-se que a resposta obtida neste trabalho contrariou, em parte, a expectativa inicial, e mostrou que o risco depende menos do formato do que de um traço específico do desenho da notação, conforme discutido adiante.

O gargalo prático de sistemas construídos sobre modelos de linguagem não é propriamente o *token*, e sim o custo, que resulta do produto entre o preço unitário do modelo e o volume que se pede a ele que produza. Usarei como exemplo dois episódios de 2026 que definem bem esse problema. A Uber esgotou em quatro meses o orçamento anual destinado a ferramentas de codificação assistida e passou a exigir um teto de US$ 1.500 mensais por funcionário e por ferramenta, diante da dificuldade, reconhecida pela própria diretoria, de ligar o consumo de *tokens* a benefício mensurável (ROPEK, 2026). No mesmo período, a Microsoft cancelou a

maior parte das licenças diretas de assistentes da Anthropic concedidas à sua engenharia e passou a rotear requisições de seus produtos para modelos próprios, declarando como objetivo explícito a redução desse custo (COOTER, 2026). Em nenhum dos casos as ações foram geradas por insuficiência de qualidade: foi o preço de mante-la em escala. Cabe notar que a parcela sobre a qual este trabalho atua é a mais cara das duas. A geração é tarifada, na prática corrente dos provedores, a um múltiplo do preço da entrada, de modo que marcação redundante **emitida** pelo modelo custa várias vezes o que custaria a mesma marcação apenas lida por ele. Sendo o custo um produto de dois fatores, há duas frentes de ataque, e a abordagem investigada aqui atua sobre ambas: emitir menos e emitir com um modelo mais barato mantendo uma qualidade adequada ao problema.

A redução do número de *tokens* nesses sistemas tem sido atacada, contudo, principalmente do lado da entrada. Na pesquisa, o LLMLingua (JIANG *et al.*, 2023) comprime o contexto antes que ele alcance o modelo, com ganhos expressivos e pouca perda de desempenho. Na indústria, ferramentas como o Headroom (Headroom Labs, 2026) aplicam a mesma ideia a saídas de ferramentas, registros e trechos recuperados, relatando reduções entre 60% e 95% em conteúdo estruturado. Essas abordagens confirmam que a economia de *tokens* é uma preocupação concreta e que boa parte dela vem da remoção de repetição estrutural. Nenhuma delas, entretanto, atua sobre o lado da geração, justamente o lado mais caro: fazer com que o próprio modelo emita nativamente uma representação compacta, cuja expansão determinística produza um artefato válido segundo um esquema formal.

Duas linhas de trabalho se aproximam desse objetivo por caminhos distintos. A decodificação restrita, discutida por Poesia *et al.* (2022), garante conformidade sintática durante a geração, mas não reduz o volume que o modelo precisa emitir. Já o ProMoAI, de Kourani *et al.* (2024), gera modelos de processo por meio de uma representação intermediária expandida deterministicamente em BPMN, demonstrando que a estratégia é viável. O trabalho de Kourani *et al.* (2024) apoia-se, contudo, em um modelo de fronteira operado por *prompt* e não trata a representação intermediária como um problema de compressão com taxa medida. É exatamente nesse espaço que esta monografia se posiciona.

A escolha do modelo é um problema também. Belcak *et al.* (2025) argumentam que modelos pequenos e especializados, podem ser uma base adequada para tarefas agênticas repetitivas no lugar de fronteira genéricos, por razões de custo, latência e previsibilidade. Uma representação intermediária suficientemente compacta e regular torna essa especialização viável: reduz o que o modelo precisa aprender a emitir e permite que até mesmo um modelo de sete

bilhões de parâmetros, como o Qwen2.5-Coder (HUI *et al.*, 2024), seja ajustado para a tarefa.

Este trabalho propõe, portanto, um protocolo de compressão semântica para geração de BPMN. Um modelo de linguagem produz uma descrição concisa do processo em uma linguagem de domínio específico projetada para esse fim, e um transpilador baseado em gramática EBNF expande essa descrição em XML BPMN 2.0 validado contra o esquema oficial. A separação de responsabilidades gera um efeito útil para a depuração: quando ocorre uma falha, ela fica localizada na geração da DSL, no transpilador ou na validação, em vez de diluída em um documento XML extenso. Medido sobre 1.021 processos com o tokenizador do Qwen2.5-Coder, o artefato proposto é **seis vezes** mais compacto que o XML lógico correspondente, equivalente a uma redução de 83,4% no número de *tokens* que o modelo precisa emitir, sobre as saídas efetivamente geradas pelos braços experimentais, a razão sobe para cerca de sete.

Os resultados contrariaram em parte as hipóteses registradas antes da execução, e por isso são antecipados aqui.

A primeira versão da notação, descrita no *prompt* de dois modelos de fronteira (DeepSeek e GLM), produziu saída válida em 64,8% e 56,6% das gerações, respectivamente, contra 98,1% e 94,3% da geração direta de XML pelos mesmos modelos. A análise dessas falhas mostrou que a maior parte delas não decorria da novidade da linguagem, e sim de um traço de seu desenho: para referenciar um elemento, a notação exigia marcá-lo com identificador no momento em que era escrito, antes de o modelo saber que um ramo posterior voltaria a ele. Um modelo autorregressivo, que gera da esquerda para a direita sem revisar o que já escreveu, não dispõe dessa informação. Acrescentada à notação a referência pelo rótulo do elemento já escrito, o mesmo modelo de fronteira passou a 86,8% de validade, com fidelidade estatisticamente indistinguível da geração direta de XML. Essa correção foi introduzida depois de observados os resultados e é reportada como emenda ao protocolo, com valor exploratório.

O modelo pequeno chega ao mesmo ponto por outro caminho. Com a notação internalizada nos pesos por ajuste fino, um modelo de sete bilhões de parâmetros atingiu os mesmos 86,8% de validade, sem precisar da gramática no contexto; o mesmo ajuste fino, repetido sobre duas famílias adicionais de modelos, produziu 83,0% em ambas, em verificação exploratória. Em fidelidade estrutural ele é inferior ao modelo de fronteira com a notação corrigida, e a troca que oferece é de custo: sob preço de mercado, cada requisição sai de duas a quatro vezes mais barata. Nenhuma das configurações alcança a validade da geração direta de XML, que permanece em 98,1%.

Registra-se também um achado de ordem metodológica, obtido durante a avaliação

e independente da linguagem proposta: foi medido a concordância entre modelos de referência produzidos por especialistas distintos para o mesmo processo, e ela é baixa. Métricas estruturais que identificam nós por rótulo saturam, por isso, muito abaixo do máximo teórico, e todos os métodos avaliados operam nessa vizinhança. Portanto, os valores absolutos dessas métricas não devem ser lidos como percentual de acerto.

1.1 MOTIVAÇÃO

A modelagem de processos ainda é caro justamente aonde é mais necessário. Empresas que documentam seus processos textualmente raramente dispõe de tempo e recurso especializado para convertê-los em modelos formais, o resultado disso é uma base documental não visual e que não pode ser analisada, simulada ou automatizada. Reduzir o custo e tornar possível a conversão amplia o acesso à modelagem e libera o especialista para focar em ações menos repetitivas e mais estratégicas.

Há ainda uma motivação técnica, e ela independe do desfecho particular observado na BPMN. O padrão de separar a geração de uma representação compacta da expansão determinística em um artefato validável é aplicável a qualquer domínio em que a saída precise obedecer a um esquema formal e a serialização oficial seja verbosa. O que este trabalho tem a oferecer a esses domínios não é apenas a confirmação de um ganho, mas a identificação da **condição** sob a qual ele se realiza: que a notação não exija do gerador antecipar o que ainda não escreveu. A BPMN oferece um caso de teste favorável para essa investigação porque possui esquema oficial, conjuntos de dados públicos com modelos de referência e métricas estruturais bem definidas, o que permite verificação objetiva em vez de avaliação subjetiva.

Por fim, a economia de *tokens* deixou de ser detalhe de implementação. Em fluxos que geram modelos em escala, o custo de emitir marcação redundante é recorrente, e a alternativa de depender continuamente de modelos de fronteira concentra esse custo em um fornecedor externo. Um modelo pequeno ajustado para emitir uma representação comprimida é, nesse cenário, uma resposta de engenharia e não apenas um exercício acadêmico.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Investigar se a geração de uma representação intermediária comprimida, seguida de expansão determinística, produz modelos BPMN 2.0 válidos de forma mais confiável e mais econômica em *tokens* do que a geração direta de XML por modelos de linguagem.

1.2.2 Objetivos Específicos

a) projetar uma linguagem de domínio específico para descrição de processos BPMN, com gramática EBNF formalmente especificada;
b) implementar um transpilador determinístico que expanda essa linguagem em XML BPMN 2.0 validado contra o esquema oficial;
c) construir um conjunto de dados de pares formados por descrição textual e representação na linguagem proposta, a partir de fontes públicas de descrições de processos;
d) realizar o ajuste fino supervisionado de um modelo de linguagem de pequeno porte para emitir a representação comprimida;
e) quantificar a compressão obtida, o custo por requisição e a fidelidade estrutural dos modelos gerados, comparando a abordagem proposta com a geração direta de XML sob protocolo de avaliação definido previamente à execução dos experimentos.

1.3 ORGANIZAÇÃO DO TRABALHO

O Capítulo 2 reúne os conceitos necessários à proposta: modelagem de processos e a notação BPMN 2.0, funcionamento e limitações dos modelos de linguagem na geração estruturada, linguagens de domínio específico como camada de mediação, e os métodos de adaptação de modelos empregados na etapa experimental.

O Capítulo 3 posiciona a proposta frente à literatura, distinguindo-a tanto das abordagens de compressão de contexto, que atuam sobre a entrada, quanto dos trabalhos que geram modelos de processo por representação intermediária sem tratá-la como problema de compressão com taxa medida.

O Capítulo **??** descreve a linguagem proposta e sua gramática, o transpilador deter-

minístico e o gerador de leiaute, o *pipeline* de construção do conjunto de dados, e o protocolo de avaliação, definido antes da execução dos experimentos, incluindo hipóteses, métricas, braços experimentais e plano de análise estatística.

O Capítulo 5 reporta a verificação do componente determinístico sobre o corpus completo e, em seguida, a avaliação comparativa dos braços experimentais contra modelos de referência produzidos por especialistas, e o custo por requisição de cada abordagem. Reporta também as emendas ao protocolo, as verificações de validade aplicadas aos resultados adversos e as limitações identificadas na própria métrica de fidelidade.

O Capítulo 6 retoma o objetivo geral à luz das evidências, sistematiza as contribuições e as limitações, e enumera as extensões que a investigação deixou em aberto.

2 FUNDAMENTAÇÃO TEÓRICA

O ponto de partida deste trabalho é uma dificuldade prática: gerar XML BPMN 2.0 completo por meio de Modelos de Linguagem de Grande Escala (LLMs, do inglês *Large Language Models*) tende a ser custoso, verboso e sujeito a erros estruturais. Para discutir esse problema, este capítulo reúne os conceitos necessários à proposta de um protocolo de compressão semântica voltado à geração de *markup* XML. A fundamentação começa pela modelagem de processos de negócio e pela notação BPMN 2.0, passa pelos princípios de funcionamento dos LLMs e pelos desafios da geração estruturada, e então discute Linguagens de Domínio Específico (DSLs, do inglês *Domain-Specific Languages*) como mecanismo de mediação entre descrição semântica e representação XML. Em seguida, são apresentados os métodos de adaptação de modelos que sustentam a etapa experimental do projeto, especialmente SFT, LoRA, QLoRA e GRPO. Por fim, o capítulo aborda recursos disponíveis para extração de processos a partir de texto natural e métricas adequadas à avaliação de artefatos estruturados.

2.1 MODELAGEM DE PROCESSOS DE NEGÓCIO E A NOTAÇÃO BPMN 2.0

2.1.1 Gerenciamento de Processos de Negócio

O Gerenciamento de Processos de Negócio (BPM, do inglês *Business Process Management*) trata processos organizacionais como objetos que podem ser descritos, analisados, executados e aperfeiçoados de forma sistemática (DUMAS *et al.*, 2018). Essa área aproxima contribuições da administração, da engenharia de software e da pesquisa operacional, sobretudo quando o processo precisa deixar de ser apenas uma prática informal e passar a funcionar como um artefato verificável (WESKE, 2019).

Essa formalização é importante porque iniciativas de automação, auditoria e conformidade dependem de uma representação compartilhada do processo. Ao mesmo tempo, a notação adotada precisa equilibrar dois requisitos que nem sempre caminham juntos: ser compreensível para analistas e gestores, sem exigir domínio técnico profundo, e preservar precisão suficiente para permitir análise computacional ou execução automatizada. A BPMN se consolidou justamente nesse espaço, oferecendo uma linguagem visual relativamente acessível, mas apoiada por uma especificação formal.

2.1.2 A Notação BPMN 2.0

A *Business Process Model and Notation*, em sua versão 2.0, é mantida pelo *Object Management Group* e é uma das principais referências para a representação de processos de negócio (Object Management Group, 2011). Seu papel é duplo: por um lado, define símbolos gráficos usados por analistas de processo; por outro, estabelece uma serialização XML descrita por um *XML Schema Definition* (XSD). Essa combinação permite que um mesmo modelo seja lido visualmente por pessoas e processado por ferramentas.

Segundo a especificação da BPMN 2.0, os elementos da notação são agrupados em categorias principais Object Management Group (2011):

- **Objetos de fluxo** (*flow objects*): eventos (*events*), atividades (*activities*) e *gateways*, que representam os principais nós do processo;
- **Objetos de conexão** (*connecting objects*): fluxos de sequência, fluxos de mensagem e associações, responsáveis por explicitar relações entre elementos;
- **Raias** (*swimlanes*): *pools* e *lanes*, usadas para indicar participantes, setores ou responsabilidades;
- **Artefatos** (*artifacts*): grupos, anotações textuais e objetos de dados, empregados para acrescentar informação complementar ao modelo.

Com esses elementos, a BPMN permite representar padrões comuns de processos, como decisões exclusivas, execuções paralelas, eventos intermediários e interações entre participantes. A expressividade da notação, no entanto, cobra seu preço: quanto mais completa é a representação, maior tende a ser a complexidade para quem modela, valida ou gera o artefato. Dumas *et al.* (2018) observam que essa riqueza da BPMN exige aprendizado, tanto do analista humano quanto de qualquer sistema que tente manipulá-la automaticamente.

2.1.3 Representação XML e Verbosidade

Na prática, um modelo BPMN 2.0 interoperável costuma ser armazenado em XML conforme o esquema oficial. Essa escolha favorece o intercâmbio entre ferramentas e reduz ambiguidades de implementação, mas produz documentos consideravelmente maiores do que a descrição conceitual do processo. Em um exemplo simples, o XML precisa declarar *namespaces*, identificadores, elementos de diagrama, coordenadas, fluxos, referências cruzadas e atributos previstos pela especificação.

Essa verbosidade não é um problema grave quando o XML é gerado por uma

ferramenta de modelagem. A situação muda quando o alvo de geração passa a ser um LLM. Nesse caso, o modelo precisa emitir uma sequência longa, com muitos trechos repetitivos e com baixa tolerância a pequenos desvios. Uma *tag* ausente, um identificador trocado ou uma referência inconsistente pode invalidar o documento inteiro, mesmo que a estrutura geral do processo tenha sido compreendida corretamente. Esse descompasso entre conteúdo semântico e representação textual motiva a busca por uma camada intermediária mais curta e mais controlável.

2.2 MODELOS DE LINGUAGEM DE GRANDE ESCALA

2.2.1 Arquitetura Transformer

A arquitetura Transformer, proposta por Vaswani *et al.* (2017), marcou uma mudança importante na modelagem de sequências. Em vez de processar a entrada de forma recorrente, o Transformer usa mecanismos de *self-attention* para relacionar diretamente diferentes posições da sequência. Esse desenho favoreceu paralelização em larga escala e abriu caminho para os modelos de linguagem atuais.

Os LLMs mais utilizados seguem, em geral, uma formulação autorregressiva: a cada passo, o modelo estima o próximo *token* a partir dos anteriores. Quando treinados sobre grandes coleções de texto, esses modelos passam a apresentar capacidade de generalização para tarefas variadas, inclusive tarefas que não aparecem explicitamente como objetivos de treinamento (BROWN *et al.*, 2020). Essa flexibilidade explica seu interesse para geração de código, consultas, marcações e outros formatos estruturados. Uma consequência dessa formulação é relevante para formatos estruturados: o modelo gera da esquerda para a direita e não revisa o que já emitiu. Cada *token* é decidido apenas com base no que o precede, de modo que um formato que exija declarar, num ponto do texto, informação que só se determina adiante impõe ao gerador uma antecipação que a decodificação não oferece.

2.2.2 Tokenização e o Custo de Sequências Longas

LLMs não operam diretamente sobre palavras completas. A entrada e a saída são decompostas em *tokens*, frequentemente obtidos por técnicas como *Byte-Pair Encoding* (SENNRICH; HADDOW; BIRCH, 2016). Essa estratégia funciona bem para texto natural, mas pode ser pouco econômica para formatos como XML. Identificadores longos, nomes de atributos, símbolos de marcação e padrões repetidos tendem a ser fragmentados em vários *tokens*.

O aumento no número de *tokens* tem consequências diretas. O mecanismo de atenção cresce em custo com o comprimento da sequência, o que afeta memória, tempo de treinamento e latência de inferência. Quando o modelo é acessado por meio de uma API comercial, o efeito é também financeiro: os provedores cobram por *token* de entrada e de saída, em geral com preço maior para a saída. Em um ambiente acadêmico com recursos limitados, gerar XML BPMN completo significa gastar parte relevante do orçamento computacional em trechos que são necessários para a conformidade do arquivo, mas não necessariamente para a descrição semântica do processo.

2.2.3 Geração de Saídas Estruturadas

A geração de artefatos estruturados por LLMs, como código-fonte, consultas e linguagens de marcação, tornou-se uma linha de pesquisa relevante a partir de trabalhos como o de Chen *et al.* (2021). O desafio central é que a geração probabilística não garante, por si só, validade sintática nem conformidade a um esquema externo. Em linguagem natural, um pequeno desvio pode ser tolerável; em XML, uma inconsistência local pode tornar todo o documento inválido.

Poesia *et al.* (2022) discutem esse problema no contexto de geração de código e defendem a combinação entre modelos probabilísticos e mecanismos determinísticos de restrição. A ideia é simples, mas importante para este trabalho: quando a saída precisa obedecer a regras formais, a geração livre do modelo deve ser complementada por alguma forma de controle. Esse controle pode aparecer durante a decodificação, em uma etapa posterior de validação ou por meio de uma representação intermediária projetada para ser mais simples de gerar e de verificar.

2.3 LINGUAGENS DE DOMÍNIO ESPECÍFICO E COMPRESSÃO SEMÂNTICA

2.3.1 Linguagens de Domínio Específico

Para Fowler (2010), uma Linguagem de Domínio Específico é uma linguagem computacional criada para um domínio particular, com escopo e expressividade intencionalmente limitados. Essa limitação não é uma deficiência. Pelo contrário, ela permite que a linguagem se aproxime dos conceitos do problema e reduza detalhes que seriam necessários em uma linguagem de propósito geral. Mernik, Heering e Sloane (2005) apontam benefícios recorrentes das DSLs, entre eles a elevação do nível de abstração, a redução de código repetitivo e a maior facilidade

para realizar análises estáticas.

No caso da BPMN, a serialização XML já é uma linguagem formal, mas não foi pensada para ser escrita diretamente por pessoas ou por modelos de linguagem. Ela foi concebida principalmente para intercâmbio entre ferramentas. Por isso, quando se pede a um LLM que produza XML BPMN completo, transfere-se ao modelo uma responsabilidade pouco adequada ao seu funcionamento: repetir com precisão uma grande quantidade de marcação técnica, ainda que parte dessa marcação possa ser gerada de maneira determinística.

2.3.2 Compressão Semântica como Estratégia de Geração

Neste trabalho, compressão semântica designa a redução da representação textual de um artefato sem perda do conteúdo essencial, desde que a forma completa possa ser reconstruída por uma etapa determinística. A compressão, portanto, não é tratada como objetivo isolado. Seu papel é separar duas responsabilidades: o LLM deve produzir uma descrição concisa do processo, enquanto o transpilador deve expandir essa descrição para XML BPMN 2.0 válido.

A proposta segue três etapas. Primeiro, o LLM gera uma representação intermediária em uma DSL voltada à descrição de processos BPMN. Em seguida, um transpilador baseado em gramática EBNF e regras de enriquecimento converte essa representação em XML BPMN 2.0. Por fim, o XML gerado é validado contra o XSD oficial. Quando ocorre erro, a falha tende a ficar mais localizada: pode estar na geração da DSL, no transpilador ou na validação do XML. Esse desenho facilita a depuração quando comparado à geração direta de um documento XML longo.

A motivação para essa estratégia combina três aspectos. O primeiro é computacional, pois uma saída mais curta reduz o número de *tokens* e pode baratear treinamento e inferência. O segundo é de aprendizado, já que uma representação menos redundante pode concentrar melhor o sinal supervisionado no que realmente descreve o processo. O terceiro é metodológico: a separação entre modelo e transpilador permite avaliar de forma mais clara o que foi aprendido pelo LLM e o que foi garantido por regras determinísticas.

2.4 ADAPTAÇÃO DE MODELOS DE LINGUAGEM A TAREFAS ESPECÍFICAS

2.4.1 Supervised Fine-Tuning

O ajuste supervisionado, ou *Supervised Fine-Tuning* (SFT), é uma das formas mais diretas de adaptar um LLM a uma tarefa específica. O procedimento continua o treinamento do modelo sobre pares de entrada e saída esperada, deslocando a distribuição gerada na direção do comportamento desejado (OUYANG *et al.*, 2022). Neste projeto, cada exemplo associa uma descrição textual de processo a uma representação na DSL proposta.

A qualidade do SFT depende fortemente do corpus. Em tarefas muito específicas, como gerar uma DSL nova para processos BPMN, não há um conjunto público pronto que resolva o problema de forma direta. Isso torna necessária a construção de dados por meio de conversões, curadoria e aumento de dados. Modelos mais capazes podem atuar como apoio nessa etapa, gerando pseudoexemplos que depois precisam ser filtrados e validados.

2.4.2 Ajuste com Eficiência de Parâmetros: LoRA e QLoRA

O ajuste completo de modelos com bilhões de parâmetros costuma ser inviável em computadores de uso acadêmico. Hu *et al.* (2022) propuseram o método *Low-Rank Adaptation* (LoRA) como uma alternativa mais econômica: em vez de atualizar todos os pesos do modelo, treinam-se matrizes de baixa dimensão acopladas a camadas específicas. Os pesos originais permanecem congelados, e apenas os parâmetros adicionais são ajustados.

O método reduz a quantidade de parâmetros treináveis e, consequentemente, o consumo de memória. O QLoRA, apresentado por Dettmers *et al.* (2023), amplia essa economia ao quantizar os pesos congelados do modelo base para 4 *bits*, mantendo as adaptações em maior precisão. Essa combinação é especialmente relevante para o presente estudo, pois viabiliza a inferência do modelo especializado em uma estação local com 11 GB de memória de GPU, enquanto o treinamento é executado em infraestrutura de nuvem.

2.4.3 Aprendizado por Reforço com GRPO

Embora o SFT ensine o modelo a imitar exemplos, ele não otimiza diretamente métricas externas de qualidade. Para uma tarefa como esta, algumas propriedades importantes são verificáveis por código: validade sintática, conformidade ao XSD, fidelidade semântica, coerência topológica do grafo e economia de *tokens*. Essas propriedades podem ser usadas como

sinais de recompensa em uma etapa posterior de treinamento.

Shao *et al.* (2024) introduziram o *Group Relative Policy Optimization* (GRPO) no contexto de modelos voltados a raciocínio matemático. O método deriva do PPO, mas evita manter um modelo crítico separado. Para cada *prompt*, gera-se um grupo de respostas, e a vantagem de cada resposta é estimada em relação à média do próprio grupo. Essa simplificação reduz custo de memória, característica importante em cenários com restrição de hardware.

Neste trabalho, o GRPO foi considerado como possível segunda etapa após o SFT e permanece como trabalho futuro (Seção **??**). A função de recompensa combina dimensões diferentes da qualidade esperada: sintaxe, semântica, topologia e eficiência da DSL gerada. A composição ponderada dessas dimensões permite transformar critérios de avaliação em um sinal de treinamento, sem reduzir a qualidade do processo a uma única métrica isolada.

2.5 EXTRAÇÃO DE PROCESSOS A PARTIR DE TEXTO NATURAL

2.5.1 Caracterização do Problema

A extração automática de modelos de processo a partir de texto natural é anterior ao uso recente de LLMs. Friedrich, Mendling e Puhlmann (2011), por exemplo, já investigavam a geração de modelos BPMN com apoio de técnicas tradicionais de Processamento de Linguagem Natural (PLN), como análise sintática, resolução de correferências e regras de mapeamento. Os resultados indicavam potencial, mas também deixavam claro que textos de processo podem conter ambiguidades e informações implícitas.

Aa *et al.* (2018) organizam esses desafios em dimensões como ambiguidade lexical e estrutural, variação de granularidade, lacunas de informação e possibilidade de múltiplos modelos válidos para uma mesma descrição. Essa última questão é particularmente importante: em muitos casos, não existe apenas uma modelagem correta. O que existe são representações mais ou menos adequadas a uma interpretação do texto, ao nível de detalhe escolhido e ao objetivo da modelagem.

2.5.2 Recursos e Datasets Disponíveis

O domínio ainda possui poucos *benchmarks* consolidados. Bellan *et al.* (2023) apresentaram o *Process Extraction from Text* (PET), composto por 45 descrições textuais anotadas com atividades, *gateways*, atores e relações de fluxo. O PET é relevante por oferecer anotações

em granularidade fina, úteis tanto para treinamento quanto para avaliação de tarefas de extração.

Outro recurso importante é o *PMo Dataset*, disponibilizado por Brissard, Cuppens e Zouaq (2025a). O conjunto reúne 55 modelos de processo com descrições textuais correspondentes e diferentes representações de modelo, incluindo BPMN, Mermaid, Graphviz, POWL code e XML simplificado. A presença de uma representação BPMN padronizada como referência torna o conjunto especialmente útil para avaliação. Já o dataset de Mangler e Klievtsova (2023) oferece 24 descrições textuais acompanhadas de múltiplos modelos BPMN possíveis, com pontuações atribuídas por especialistas, o que ajuda a observar a ambiguidade natural da modelagem de processos.

Além desses conjuntos especializados, o projeto pode recorrer a textos procedurais não anotados, como trechos públicos de documentação organizacional, para etapas de aumento de dados. A distinção entre essas fontes é metodologicamente relevante: datasets anotados servem como referência e avaliação, enquanto textos abertos podem alimentar a construção de pares sintéticos, desde que passem por validação.

2.6 AVALIAÇÃO DE GERAÇÃO DE ARTEFATOS ESTRUTURADOS

2.6.1 Validação por Esquema

A validação por esquema é o primeiro critério para avaliar XML BPMN. No caso da BPMN 2.0, o XSD oficial define se o documento respeita a estrutura esperada pela especificação. O resultado é binário: o XML é válido ou inválido. Esse critério é necessário, mas não suficiente. Um arquivo pode ser válido em termos de esquema e ainda assim representar um processo diferente daquele descrito no texto de entrada.

2.6.2 Métricas Sintáticas e Semânticas

Para avaliar a fidelidade ao processo de referência, métricas como precisão, revocação e F1 podem ser calculadas sobre elementos extraídos do modelo gerado. Atividades, atores, *gateways* e arestas podem ser comparados com os elementos presentes no modelo esperado. A medida F1 é útil nesse contexto porque resume precisão e revocação, evitando uma leitura baseada apenas em acertos brutos.

2.6.3 Distância de Edição em Grafos

Como modelos BPMN podem ser interpretados como grafos, a comparação estrutural também é necessária. A *Graph Edit Distance* (GED), associada ao trabalho de Sanfeliu e Fu (1983), mede a distância entre grafos a partir do custo mínimo de operações de edição, como inserção, remoção e substituição de vértices e arestas. Em formulações gerais, o cálculo exato é computacionalmente difícil, razão pela qual aproximações são frequentemente usadas em aplicações práticas (GAO *et al.*, 2010).

Para este estudo, a GED é interessante porque considera a organização topológica do processo, não apenas a presença ou ausência de elementos individuais. Ainda assim, sua utilidade depende do modelo de custo adotado. Esse modelo precisa refletir o que, no contexto da BPMN, deve ser tratado como diferença relevante entre dois processos.

2.6.4 Eficiência: Razão de Compressão de Tokens

A Razão de Compressão de Tokens (TCR, do inglês *Token Compression Ratio*) é usada para medir a eficiência da representação intermediária. A métrica compara a quantidade de *tokens* necessária para representar o processo em XML BPMN com a quantidade necessária para representá-lo na DSL proposta, usando o mesmo tokenizador. Valores superiores a um indicam que a DSL é mais compacta que o XML correspondente.

Essa métrica, sozinha, não mede qualidade. Uma representação pode ser muito curta e ainda assim perder informação relevante. Por isso, a TCR precisa ser analisada junto às métricas de validade, fidelidade semântica e estrutura topológica. O interesse do trabalho está exatamente nesse equilíbrio: reduzir a saída gerada pelo LLM sem comprometer a reconstrução determinística do BPMN.

2.6.5 Função de Recompensa Composta

No treinamento por GRPO, as métricas anteriores podem ser combinadas em uma função de recompensa composta. Essa função precisa equilibrar critérios diferentes: validade sintática, adequação semântica ao texto, plausibilidade topológica e eficiência em *tokens*. Uma formulação ponderada facilita a interpretação dos resultados, pois permite identificar que dimensão da qualidade está contribuindo mais ou menos para o comportamento do modelo.

2.7 SÍNTESE DO CAPÍTULO

A discussão apresentada neste capítulo delimita o problema e justifica a estratégia proposta. A BPMN 2.0 oferece uma representação rica para processos de negócio, mas sua serialização XML é longa e sensível a erros. LLMs são capazes de gerar texto estruturado, porém não garantem conformidade formal por si mesmos. DSLs oferecem uma alternativa intermediária: descrevem o conteúdo essencial em uma forma mais curta, enquanto um transpilador assume a expansão para XML válido. As técnicas de SFT, LoRA, QLoRA e GRPO fornecem o caminho para adaptar o modelo à tarefa, e as métricas discutidas permitem avaliar a solução sob diferentes aspectos.

Com essa base, o Capítulo de Metodologia pode especificar de maneira operacional a gramática da DSL, o funcionamento do transpilador, a construção dos dados, os parâmetros de treinamento e o protocolo de avaliação.

3 TRABALHOS RELACIONADOS

Este capítulo situa o trabalho em relação a quatro linhas de pesquisa que o tangenciam: geração de modelos de processo por modelos de linguagem, representações compactas de processos, compressão de *tokens* em sistemas com LLMs, e garantia de validade sintática na geração de linguagens formais. O critério de organização é a proximidade com a pergunta desta pesquisa — o custo de gerar marcação hierárquica verbosa —, e não a cronologia ou a autoria.

3.1 GERAÇÃO DE MODELOS DE PROCESSO COM LLMS

A referência mais próxima em objetivo é o ProMoAI (KOURANI *et al.*, 2024), que emprega um LLM de fronteira, guiado por *prompt*, para produzir código POWL executável, posteriormente traduzido para BPMN. A estratégia compartilha com este trabalho a premissa central de **não** pedir XML diretamente ao modelo, interpondo uma representação intermediária mais adequada às capacidades do gerador.

As diferenças são de natureza e de escopo. O ProMoAI depende de um modelo de fronteira em tempo de inferência, o que mantém o custo por geração no patamar que este trabalho procura reduzir; a representação intermediária é código Python, escolhida por familiaridade do modelo com tarefas de programação, e não projetada para minimizar *tokens* emitidos; e a compressão obtida não é medida nem tratada como objetivo. Aqui, ao contrário, a compressão é métrica de primeira classe, a representação é projetada para isso, e o gerador-alvo é um modelo pequeno especializado por ajuste supervisionado. A reexecução do ProMoAI está fora do escopo desta versão, de modo que ele é tratado como referência qualitativa; comparar com números publicados sob protocolo distinto seria metodologicamente indefensável.

3.2 REPRESENTAÇÕES COMPACTAS DE PROCESSOS

Diversas notações alternativas ao XML BPMN foram propostas com objetivos que se sobrepõem parcialmente ao deste trabalho. A representação *JSON branches* (KÖPKE; SAFAN, 2025) foi introduzida explicitamente para reduzir contagem de *tokens* e favorecer aderência a esquema na geração por LLMs. A representação *Process Model Elements* (VOELTER *et al.*, 2024) organiza o modelo em listas planas de elementos, facilitando contagem e comparação. Somam-se a elas notações de visualização como Mermaid e Graphviz, e simplificações do próprio XML.

O trabalho de Brissard, Cuppens e Zouaq (2025b) é o que mais se aproxima em objeto empírico: realiza análise comparativa de nove representações de modelos de processo para geração com LLMs, sobre o mesmo conjunto de dados aqui utilizado como *holdout*. É, por isso, o antecedente que exige posicionamento mais cuidadoso.

A distinção é de **pergunta de pesquisa**, e não de artefato. Aquele trabalho pergunta qual representação produz os melhores modelos de processo quando um LLM é instruído por *prompt* — uma pergunta sobre adequação de notação à tarefa de modelagem. Esta pesquisa pergunta se a estratégia de comprimir a saída e expandi-la deterministicamente torna economicamente viável a geração de marcação hierárquica verbosa por modelos pequenos, usando BPMN como caso de estudo instrumentado. Disso decorrem três diferenças operacionais:

a) **o gerador**: representações avaliadas por *prompt* em modelos de fronteira, contra um modelo pequeno especializado por ajuste supervisionado, cujo custo por requisição é de duas a quatro vezes menor sob preço de mercado (Seção **??**);
b) **a garantia sintática**: notações comparadas quanto à qualidade do modelo produzido, contra uma cadeia em que a validade perante o esquema é assegurada por construção sempre que a DSL é aceita pela gramática, o que desloca a variável aleatória do documento inteiro para a aceitação de uma representação curta;
c) **a métrica de custo**: compressão medida e reportada como resultado de primeira ordem, e não como propriedade qualitativa da notação.

Que representações compactas de BPMN já existam não é objeção a este trabalho — é evidência de que o problema é reconhecido. A contribuição pretendida não é a notação em si, mas a demonstração empírica de que a combinação entre compressão medida, expansão determinística verificável e especialização de modelo pequeno desloca a fronteira de custo da tarefa. Registre-se ainda que a DSL proposta cobre construções que algumas dessas notações não representam — múltiplos eventos de início e fim, fluxos de mensagem e raias —, embora essa maior expressividade seja consequência do desenho, e não sua justificativa. Os resultados acrescentam a essa comparação uma condição de desenho que as notações citadas não explicitam: a forma de referência entre elementos deve permitir apontar para o que já foi escrito, sem exigir do gerador antecipar o que ainda não escreveu (Seção **??**).

3.3 COMPRESSÃO DE *TOKENS*: ENTRADA *VERSUS* SAÍDA

A redução do consumo de *tokens* em sistemas com LLMs é problema ativo, e a literatura predominante trata do lado da **entrada**. O LLMLingua (JIANG *et al.*, 2023) comprime *prompts* removendo conteúdo de baixa informação segundo um modelo de linguagem auxiliar, preservando o desempenho na tarefa a jusante. Trabalhos de gestão de contexto perseguem objetivo análogo por seleção e sumarização do que entra na janela.

Este trabalho ataca o lado oposto e complementar: o custo da **saída**. A assimetria é relevante e raramente explicitada. *Tokens* de entrada podem ser processados em paralelo e beneficiam-se de mecanismos de cache; *tokens* de saída são gerados sequencialmente, dominam a latência percebida e são tipicamente tarifados a preço superior. Em tarefas cuja saída é marcação verbosa, o gargalo econômico está na geração, não na leitura — e comprimir o *prompt* não o alivia. As duas famílias de técnica são, portanto, ortogonais e combináveis, e não concorrentes. Os resultados mostram, porém, que a fronteira entre os dois lados não é estanque: quando a notação compacta é ensinada por *prompt*, a gramática passa a dominar o custo da entrada, e o ajuste fino, ao dispensá-la, atua também sobre esse lado (Seção **??**).

3.4 MODELOS PEQUENOS ESPECIALIZADOS

A viabilidade da proposta depende de que um modelo de pequeno porte, especializado, alcance desempenho competitivo em tarefa estreita. Essa premissa é sustentada por evidência recente sobre o papel de modelos pequenos em sistemas agênticos (BELCAK *et al.*, 2025), que argumenta serem eles suficientes — e economicamente preferíveis — para subtarefas repetitivas e bem delimitadas, reservando modelos de fronteira para as etapas que exigem generalidade. Os resultados deste trabalho confirmam a premissa em validade sintática, em que o modelo pequeno especializado iguala o de fronteira, mas não em fidelidade estrutural, em que permanece inferior (Seção **??**).

A geração de marcação estruturada a partir de descrição textual é exatamente uma dessas subtarefas: formato fixo, vocabulário restrito, critério de sucesso verificável. Métodos de ajuste eficiente em parâmetros (HU *et al.*, 2022), especialmente em sua variante quantizada (DETTMERS *et al.*, 2023), tornam a especialização exequível com recursos computacionais modestos, o que é condição para que o argumento econômico se sustente.

3.5 GARANTIA DE VALIDADE SINTÁTICA

Há uma alternativa à estratégia adotada que precisa ser reconhecida: a **decodificação restrita**, que impõe a gramática do formato-alvo durante a amostragem, impedindo por construção a emissão de sequências inválidas (POESIA *et al.*, 2022). Aplicada ao XML BPMN, garantiria validade sintática sem representação intermediária alguma.

Duas considerações delimitam a comparação. A primeira é que a decodificação restrita resolve a validade, mas **não** o custo: o modelo continua emitindo cada *token* da marcação verbosa, de modo que a economia perseguida aqui permanece inalcançada — e, com efeito, as duas técnicas são combináveis, podendo a decodificação restrita ser aplicada sobre a própria DSL. A segunda é que restringir a decodificação a uma gramática garante boa formação sintática, mas não coerência estrutural: um documento pode respeitar o esquema e ainda referenciar identificadores inexistentes ou descrever um grafo desconexo. A expansão determinística não impede que o gerador escreva esse erro, mas o detecta e o rejeita: uma referência a elemento inexistente no fluxo é falha de transpilação, e não documento válido e incoerente. Essa foi, aliás, a classe de falha mais frequente nos braços de DSL (Seção **??**).

A avaliação empírica da decodificação restrita como braço experimental está fora do escopo desta versão, e é registrada como trabalho futuro derivado.

4 METODOLOGIA

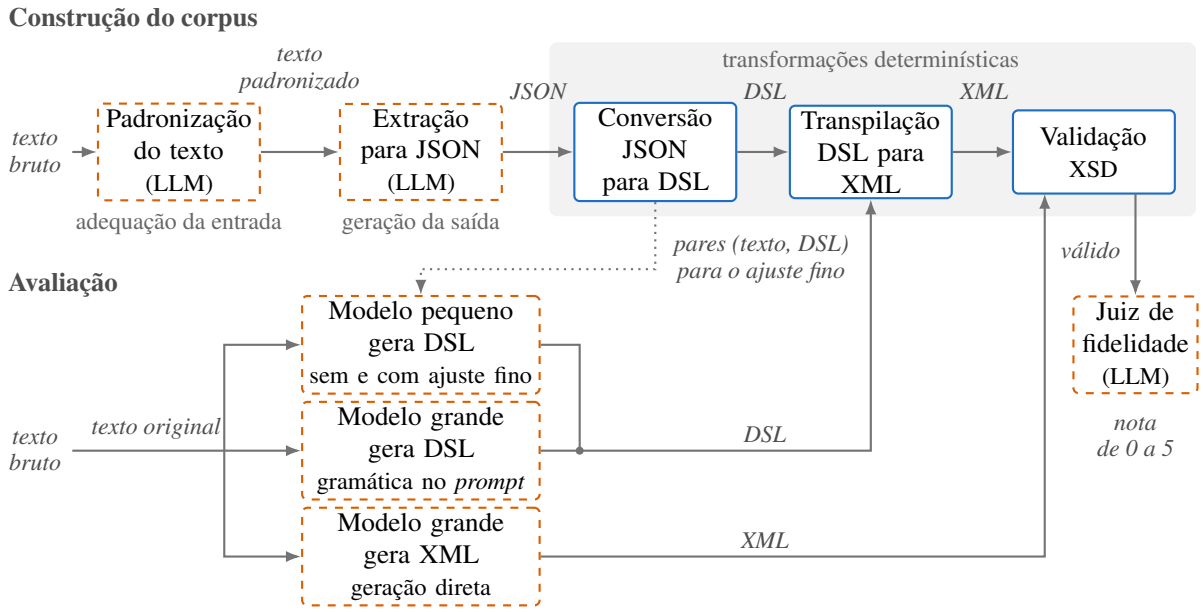
4.1 VISÃO GERAL

Neste trabalho, o modelo de linguagem não escreve o XML do BPMN 2.0 diretamente. Ele gera uma representação compacta do processo, a BPMN-DSL, e um programa determinístico, o transpilador, expande essa representação para o XML completo. O experimento compara essa estratégia com a geração direta de XML e verifica se um modelo de pequeno porte, especializado na DSL, alcança os resultados de modelos de grande porte com uma fração do uso de tokens e do custo.

O trabalho tem duas fases, resumidas na Figura 1. Na construção do corpus, descrições textuais de processos, a maior parte extraída do *handbook* público da GitLab, são padronizadas e depois convertidas em um JSON que descreve o processo, em duas chamadas a um modelo de grande porte. Um conversor determinístico transforma esse JSON na DSL, e o transpilador gera o XML correspondente (Seção 4.2). Os pares formados pelo texto original e pela DSL treinam o modelo pequeno. Trata-se de uma destilação: o modelo pequeno aprende a partir dos dados que os modelos grandes produziram (Seção 4.5). Na avaliação, cada braço experimental recebe o texto de um processo do conjunto de teste e gera XML ou DSL. A DSL passa pelo transpilador, e todas as saídas são medidas da mesma forma (Seções 4.6 e 4.7).

A divisão de responsabilidades entre as partes define o que precisa ser medido. O transpilador garante que toda DSL sintaticamente correta produza um XML válido segundo o esquema oficial do BPMN 2.0, o que foi verificado nas 1.021 amostras do corpus (Seção 4.4). O modelo continua responsável pela sintaxe da DSL e pelo conteúdo do processo, e pode errar em qualquer um dos dois. Por isso a validade e a fidelidade são medidas separadamente. A validade é verificada por programa, contra o esquema XSD. A fidelidade, isto é, se o modelo representa o processo descrito no texto, exige julgamento e foi medida por um modelo de linguagem usado como juiz. Como a validação desse juiz foi apenas parcial (Seção 4.7), as conclusões principais se apoiam na validade e no custo, e a fidelidade é reportada como exploratória.

Figura 1 – Visão geral: construção do corpus e avaliação



Fonte**:** Elaborado pelo autor

Nota**:** Borda tracejada indica etapa probabilística; borda sólida, etapa determinística. Na construção do corpus, a padronização só adequa o texto de entrada, e a extração gera a saída como JSON; daí em diante há apenas transformações determinísticas entre representações. Os pares formados pelo texto original e pela DSL treinam o modelo pequeno (seta pontilhada). Na avaliação, os braços recebem o texto original e ocupam o lugar da extração: os que geram DSL passam pela mesma transpilação, e os que geram XML seguem direto para a validação. As saídas válidas seguem para o juiz de fidelidade, um modelo de linguagem que compara o processo gerado com o texto original; por isso ele também tem borda tracejada.

4.2 DADOS

4.2.1 Fontes

Não existe um conjunto público, em escala e confiável, que associe descrições textuais de processos a modelos BPMN. Dado esse fato, somente o lado da entrada foi coletado e o lado da saída foi gerado pelo processo descrito na Seção 4.1. A Tabela 1 resume as fontes e o uso de cada uma.

Tabela 1 – Fontes de dados e uso no trabalho

Fonte	Itens	Uso
Handbook da GitLab, seções com decisões ou atores	728	treino
Handbook da GitLab, seções lineares	172	só verificação do transpilador
PET (BELLAN *et al.*, 2023)	44	treino
PMo (BRISSARD; CUPPENS; ZOUAQ, 2025a)	53	avaliação ($n = 46$, ver Seção 4.2.3)
Zenodo (MANGLER; KLIEVTSOVA, 2023)	24	validação da medida de fidelidade

Fonte**:** Elaborado pelo autor

A principal fonte de volume foi o *handbook* público da GitLab que é um manual operacional aberto com milhares de páginas. As páginas foram coletadas e divididas em seções, e cada seção recebeu uma pontuação procedural baseada na presença de passos numerados, decisões e atores. Foram mantidas as seções com passos e com decisões ou atores, que descrevem processos com alguma estrutura além de uma lista. As seções lineares, só com passos, foram separadas e não entram nos experimentos. O PET (BELLAN *et al.*, 2023) também entra no treino com 44 descrições de processos de outros domínios.

O conjunto de teste é o PMo (BRISSARD; CUPPENS; ZOUAQ, 2025a), que reúne 53 descrições de processos de fontes distintas, cada uma acompanhada de um modelo BPMN de referência. O Zenodo (MANGLER; KLIEVTSOVA, 2023) traz 24 descrições, as mesmas de 24 itens do PMo, cada uma com 8 a 11 modelos feitos por modeladores iniciantes e avaliados por um especialista com uma nota de 0 a 5. Essas notas são usadas na validação da medida de fidelidade (Seção 4.7). A sobreposição entre as duas fontes tem uma consequência para as referências do PMo: em pelo menos 10 desses 24 itens, a referência coincide, após normalização dos rótulos e fluxos, com um dos modelos feitos por iniciantes no Zenodo, e não é, portanto, um modelo de especialista. Por isso, a fidelidade das saídas é julgada contra o texto, e não contra a referência (Seção 4.7).

4.2.2 Geração dos pares de treino

Cada descrição de treino passa por duas chamadas a um modelo de grande porte, o GLM 5.2. A primeira padroniza o texto: reorganiza a descrição em atores, passos, decisões e atividades paralelas, sem acrescentar informação. A segunda converte o texto padronizado em um JSON com os elementos, os fluxos e as raias do processo. Nos casos em que o GLM 5.2 falhou na geração do JSON, foi usado o Kimi K2.6 como alternativa, o que ocorreu em 32 das 1.021 amostras.

A escolha do GLM 5.2 veio de um teste piloto com 50 amostras de treino, que comparou três configurações: o GLM 5.2 nas duas etapas, a combinação de Kimi K2.6 e DeepSeek V4 Pro, e o DeepSeek V4 Flash. O GLM 5.2 teve a maior taxa de JSON aproveitável e o menor número de falhas, além da maior proporção de XML válido ao fim do processo (96%, contra 92% da segunda melhor configuração). Essa vantagem, porém, corresponde a apenas 2 amostras em 50, e o próprio piloto mostrou que repetir a geração de uma mesma amostra pode mudar o resultado. Por isso, a escolha se apoia no fato de o GLM 5.2 ter ficado à frente nas três

medidas ao mesmo tempo, e não em uma diferença estatisticamente comprovável.

O JSON é convertido em DSL por um programa determinístico, e a DSL é transpilada para XML. Para o treino, só foram mantidos os pares em que o XML reconstrói exatamente a topologia do JSON de origem, critério atendido por 768 das 772 amostras de treino. Os pares de treino são formados pelo texto original e pela DSL. O texto padronizado serve apenas para gerar os dados e não é visto pelo modelo pequeno. Todos os rótulos dos artefatos gerados são em inglês, a mesma língua dos textos de origem e do conjunto de teste.

4.2.3 Partições e prevenção de contaminação

Os 768 pares foram divididos em treino (690) e validação (78). A divisão é feita por documento de origem, de modo que seções da mesma página do *handbook* ficam sempre do mesmo lado. Como as seções de uma página compartilham redação e vocabulário, uma divisão aleatória colocaria seções quase iguais dos dois lados, e a perda de validação mediria memorização, e não generalização.

Os conjuntos de avaliação, PMo e Zenodo, por também terem modelos BPMN de referência já em suas fontes, foram escolhidos para o conjunto de testes, e nunca entram no treino. Porém, uma verificação posterior mostrou que 7 dos 53 itens do PMo têm como texto descrições que também fazem parte do PET: seis vêm do subconjunto PET-7 incorporado ao PMo, e um sétimo coincide com um documento do PET. Como o PET está no treino, esses 7 itens foram excluídos da avaliação, que passa a ter 46 itens.

Para verificar se havia outras sobreposições, cada texto de avaliação foi comparado com os 944 textos da partição de treino, contando com as seções lineares do *handbook*, por sequências de oito palavras consecutivas, após normalização de caixa e pontuação. Os 7 itens excluídos compartilham entre 87% e 100% dessas sequências com um documento do PET. Nenhum dos outros 70 textos de avaliação (46 do PMo e 24 do Zenodo) compartilha sequência alguma. Com sequências de cinco palavras, a maior sobreposição entre eles é de cerca de 2%, formada por expressões comuns da língua, como “if you do not have” e “at the end of the”.

Uma limitação desta composição é que 94,4% dos pares de treino vêm do *handbook* da GitLab, de uma única empresa e com estilo próprio, enquanto o teste é feito sobre processos de outros domínios. Isso é discutido na Seção 4.10.

4.3 A BPMN-DSL

A BPMN-DSL é a representação intermediária, criada de forma autoral, para este trabalho. Existem outras representações textuais de processos, comparadas por Brissard, Cuppens e Zouaq (2025b), mas o foco aqui é específico: uma notação que um modelo de linguagem consiga gerar da esquerda para a direita, com poucos *tokens*, e que tenha uma tradução determinística e completa para BPMN 2.0 XML.

O Código 1 mostra um processo com uma decisão e uma repetição usando o BPMN-DSL, e o Código 2 mostra o XML que o transpilador gera a partir dele. A DSL tem 286 caracteres e o XML, 1.833.

A diferença de tamanho se dá principalmente pela forma de ligar os elementos. No XML, cada ligação é um elemento `sequenceFlow` próprio, que repete os identificadores da origem e do destino, e esses identificadores precisam ser consistentes em todo o documento. Na DSL, a ligação é dada pela posição: o operador `->` liga um elemento ao seguinte, e os ramos de uma decisão ficam dentro do bloco dela. A junção dos ramos também é implícita, e o transpilador cria o *gateway* de convergência quando ele é necessário. A razão de compressão medida no corpus e nas saídas dos braços é apresentada na Seção 4.7.

O desenho da notação segue quatro princípios:

1. cada conceito do BPMN tem uma única forma de escrita, o que reduz as escolhas que o modelo precisa fazer; a exceção é a referência, que tem duas formas, descritas a seguir;
2. o nome de um elemento serve à leitura, e o identificador, à referência, de modo que só recebem identificador os elementos referenciados em outro ponto;
3. raias são escopos que envolvem parte do fluxo (bloco `@lane`), em vez de listas de elementos declaradas à parte, o que evita declarar cada atividade duas vezes;
4. ramos vazios de uma decisão são escritos explicitamente, com `()`.

Um elemento que já apareceu no fluxo é referenciado com `#`, o que permite escrever repetições e convergências. Existem duas formas de referência, equivalentes para o transpilador. Na primeira, o elemento recebe um identificador quando aparece (`service "Fetch data" #fetch`) e é referenciado por ele (`#fetch`), como no Código 1. Na segunda, a referência repete o rótulo do elemento (`#"Fetch data"`), sem declaração prévia. A comparação de rótulos ignora diferenças de caixa, espaçamento e pontuação, e só alcança elementos que já apareceram. As duas formas produzem o mesmo XML. A diferença entre elas é para quem gera: a primeira exige decidir, ao

Código-fonte 1 – Processo com decisão e repetição na BPMN-DSL

```
process "Order Processing with Retry" {
  start
  -> service "Fetch data" #fetch
  -> service "Validate"
  -> xor "Valid?" {
    [yes] -> service "Process" -> end
    [no]  -> xor "Retry?" {
               [yes] -> #fetch
               [no]  -> end:error "Invalid"
             }
  }
}
```

Código-fonte 2 – XML gerado pelo transpilador a partir do Código 1

```
<definitions xmlns="http://www.omg.org/spec/BPMN/20100524/MODEL" ...>
  <process id="Process_Order_Processing_with_Retry" name="Order Processing
      with Retry">
    <startEvent id="StartEvent_1" name="start"/>
    <serviceTask id="ServiceTask_1" name="Fetch data"/>
    <sequenceFlow id="SequenceFlow_1" sourceRef="StartEvent_1" targetRef="
        ServiceTask_1"/>
    <serviceTask id="ServiceTask_2" name="Validate"/>
    <sequenceFlow id="SequenceFlow_2" sourceRef="ServiceTask_1" targetRef="
        ServiceTask_2"/>
    <exclusiveGateway id="ExclusiveGateway_1" name="Valid?"/>
    <serviceTask id="ServiceTask_3" name="Process"/>
    <sequenceFlow id="SequenceFlow_3" sourceRef="ExclusiveGateway_1"
        targetRef="ServiceTask_3">
      <conditionExpression>yes</conditionExpression>
    </sequenceFlow>
    <endEvent id="EndEvent_1" name="end"/>
    <sequenceFlow id="SequenceFlow_4" sourceRef="ServiceTask_3" targetRef="
        EndEvent_1"/>
    <exclusiveGateway id="ExclusiveGateway_2" name="Retry?"/>
    <sequenceFlow id="SequenceFlow_5" sourceRef="ExclusiveGateway_2"
        targetRef="ServiceTask_1">
      <conditionExpression>yes</conditionExpression>
    </sequenceFlow>
    <endEvent id="EndEvent_2" name="Invalid">
      <errorEventDefinition/>
    </endEvent>
    <sequenceFlow id="SequenceFlow_6" sourceRef="ExclusiveGateway_2"
        targetRef="EndEvent_2">
      <conditionExpression>no</conditionExpression>
    </sequenceFlow>
    <sequenceFlow id="SequenceFlow_7" sourceRef="ExclusiveGateway_1"
        targetRef="ExclusiveGateway_2">
      <conditionExpression>no</conditionExpression>
    </sequenceFlow>
    <sequenceFlow id="SequenceFlow_8" sourceRef="ServiceTask_2" targetRef="
        ExclusiveGateway_1"/>
  </process>
</definitions>
```

escrever o elemento, que ele será referenciado mais adiante; a segunda não.

A DSL cobre eventos iniciais, intermediários e finais, com gatilhos de mensagem, temporizador, erro, sinal e escalonamento; oito tipos de tarefa; *gateways* exclusivos, paralelos, inclusivos e baseados em eventos; subprocessos; chamadas a outros processos; raias; *pools* com fluxos de mensagem entre eles; e anotações. Ficam fora desta versão os eventos de borda, a compensação e as atividades de múltiplas instâncias.

4.4 TRANSPILAÇÃO

Existem dois programas determinísticos que fazem as conversões entre representações. O primeiro converte o JSON do processo para DSL e é usado só na construção do corpus. O segundo é o transpilador, ele transforma a DSL em BPMN 2.0 XML e é usado tanto no corpus quanto na avaliação.

A conversão de JSON para DSL percorre o grafo do processo e segue uma regra: todo elemento alcançável é escrito uma única vez, e qualquer novo encontro com um elemento já escrito vira uma referência. Essa regra é o que permite representar laços, ramos que voltam a um ponto anterior e convergências de vários ramos sem duplicar elementos nem perder ligações. O conversor sempre usa a referência por identificador, de modo que o corpus de treino contém só essa forma.

O transpilador lê a DSL com um analisador gerado a partir da gramática formal da notação e traduz cada construção em um elemento XML fixo. Cada ligação vira um `sequenceFlow`; um bloco de decisão vira o *gateway* de divisão e, quando algum ramo continua depois dele, o *gateway* de convergência; um bloco `@lane` vira a raia com a lista dos seus elementos; um *pool* vira um participante, com seus fluxos de mensagem. Um ramo que termina em um evento de fim ou em uma referência não é ligado à convergência, porque ali o fluxo já se encerrou ou seguiu para outro ponto. Os identificadores do XML são gerados pelo transpilador e são únicos no documento.

O componente foi verificado de duas formas sobre as 1.021 amostras do corpus. Na primeira, o XML de todas as amostras foi validado contra o esquema XSD oficial do BPMN 2.0, e as 1.021 passaram. Na segunda, a estrutura do XML foi comparada com a do JSON originalmente gerado, pelos pares de atividades e eventos que se sucedem no fluxo. As estruturas ficaram idênticas em 1.020 amostras. Na amostra restante, o próprio JSON tem fluxos saindo de eventos de fim, que o transpilador não reproduz porque um evento de fim encerra o fluxo.

Nas saídas dos braços experimentais, as 951 DSL que seguiram a gramática produziram XML válido. A falha de validade de uma saída em DSL vem, portanto, sempre da geração, e não da transpilação.

O XML gerado contém só a lógica do processo. Para que seja possível abrir o modelo em uma ferramenta de modelagem, é necessário possuir um pós-processador que acrescenta as coordenadas de cada elemento, com um leiaute em camadas no estilo de Sugiyama, Tagawa e Toda (1981): os laços são desconsiderados para ordenar os elementos, cada elemento é posicionado pela distância até o início, e as raias são respeitadas. O leiaute é determinístico e serve apenas à visualização. Nenhuma medida deste trabalho depende dele: a validade e a razão de compressão são medidas sobre o XML sem coordenadas.

4.5 ESPECIALIZAÇÃO DO MODELO

O modelo pequeno é o Qwen2.5-Coder-7B-Instruct (HUI *et al.*, 2024), tem 7 bilhões de parâmetros e licença Apache 2.0 e foi ajustado de forma supervisionada sobre os pares de texto original como entrada e DSL descritos na Seção 4.2 como saída. O texto original foi mantido apesar de existir uma versão do texto padronizado (Seção 4.2.2) porque os textos do conjunto de teste também são descrições sem padronização textual. As saídas em DSL foram geradas por modelos de grande porte, por isso o ajuste é uma destilação: o modelo pequeno aprende a reproduzir, na DSL, o que os modelos grandes extraíram dos textos. Ele não vê o texto padronizado, o JSON intermediário nem o BPMN 2.0 XML final.

O ajuste usa QLoRA (DETTMERS *et al.*, 2023). Os pesos do modelo base ficam congelados e quantizados em 4 bits, e só são treinadas matrizes de baixo posto acrescentadas às camadas lineares (HU *et al.*, 2022). A perda é calculada apenas sobre os *tokens* da DSL, e não sobre os do *prompt*. O *prompt* de treino é o mesmo usado na avaliação do modelo ajustado: curto e sem a gramática da DSL, de modo que o modelo aprende a notação pelos exemplos (o *prompt* completo está no Apêndice A). Os hiperparâmetros estão na Tabela 2.

Dos 690 pares de treino, 18 passam de 4.096 *tokens* e foram descartados, restando 672. O treino foi feito em uma GPU A40 alugada, com 48 GB de VRAM, em 1 hora e 1 minuto, ao custo de US$ 0,47. A perda na validação foi medida ao fim de cada época: 0,440, 0,416 e 0,422. O adaptador mantido é o da segunda época, a de menor perda. Como a validação é separada por documento de origem (Seção 4.2.3), essa escolha favorece a generalização para documentos não vistos, e não a memorização.

Tabela 2 – Configuração do ajuste supervisionado

Parâmetro	Valor
Modelo base	Qwen2.5-Coder-7B-Instruct
Quantização do modelo base	4 bits (NF4, dupla quantização)
Posto e α do LoRA	16 e 32
Dropout do LoRA	0,05
Camadas adaptadas	todas as projeções lineares
Comprimento máximo da sequência	4.096 *tokens*
Épocas	3
Taxa de aprendizado	10^{-4}, cosseno, 3% de aquecimento
Lote efetivo	8 (1 exemplo, 8 passos de acumulação)
Otimizador	AdamW paginado de 8 bits
Semente	42

Fonte**:** Elaborado pelo autor

Na avaliação, o modelo roda localmente em uma RTX 2080 Ti de 11 GB de VRAM, com o modelo base em 4 bits, cálculos em 16 bits de ponto flutuante e decodificação gulosa, com até 2.048 *tokens* gerados. O braço de controle com o mesmo modelo sem o adaptador (Seção 4.6) usa exatamente a mesma configuração, para que a diferença entre os dois meça o efeito do ajuste, e não da forma de execução. O treino usou o formato *bfloat16*, disponível na A40, e a inferência usa *float16*, porque a arquitetura da 2080 Ti não oferece o primeiro.

4.6 BRAÇOS EXPERIMENTAIS

Cada braço experimental é uma forma de gerar o modelo BPMN a partir da descrição: um modelo de linguagem, um *prompt* e um formato de saída. Todos os braços recebem as mesmas descrições do conjunto de teste, no texto original. Os braços que emitem DSL passam pelo mesmo transpilador (Seção 4.4), e os que emitem XML são avaliados diretamente. A Tabela 3 resume os braços.

Os sete primeiros braços foram definidos e congelados antes da primeira execução. O A1 e o A1g representam a prática corrente, em que o modelo de grande porte escreve o BPMN 2.0 XML diretamente, e contempla comparação externa ao método. O A2 usa o mesmo modelo, mas pede a DSL, de modo que a diferença entre A2 e A1 vem só do formato de saída. O DeepSeek V4 Pro foi escolhido por não ter participado da geração do corpus. O par A1g e A2g repete a mesma comparação com o GLM 5.2, que gerou o corpus. Se o efeito do formato aparecer com um modelo e não com o outro, indica que o efeito está atrelado ao mesmo, e não da DSL. A2 e A2g não são concorrentes do método: são o próprio método executado com um modelo de grande porte, a referência que o modelo pequeno tenta alcançar.

Tabela 3 – Braços experimentais

Braço	Modelo	Saída	*Prompt*	O que o braço mede
A1	DeepSeek V4 Pro	XML	XML	a prática corrente: XML gerado direto
A2	DeepSeek V4 Pro	DSL	gramática	o efeito do formato, com o modelo fixo
A1g	GLM 5.2	XML	XML	A1 com o modelo que gerou o corpus
A2g	GLM 5.2	DSL	gramática	A2 com o modelo que gerou o corpus
A3	Qwen 7B	DSL	gramática	o modelo pequeno sem ajuste
A3m	Qwen 7B	DSL	mínimo	o controle do A4: só falta o adaptador
A4	Qwen 7B†	DSL	mínimo	o modelo pequeno especializado
A2r	DeepSeek V4 Pro	DSL	gramática com rótulo	se as falhas do A2 vêm das referências
X-lc25/50	Qwen 7B†	DSL	mínimo	o A4 treinado com 25% e 50% dos dados
X-gr	Granite 4.1 8B†	DSL	mínimo	o A4 em outra família de modelo
X-mi	MiMo-7B-RL†	DSL	mínimo	o A4 em outra família de modelo

Fonte**:** Elaborado pelo autor

Nota**:** Qwen 7B é o Qwen2.5-Coder-7B-Instruct; † com o adaptador treinado (Seção 4.5). Os braços acima do traço foram definidos antes de qualquer execução; os de baixo são exploratórios (ver texto).

Os três braços do modelo pequeno separam o efeito do ajuste do efeito da instrução. O A3 recebe o *prompt* com a gramática completa, o mesmo do A2, e mede o que o modelo pequeno consegue sem ajuste. O A4 recebe o *prompt* mínimo, sem gramática nem exemplo da notação, que é o mesmo usado no treino (Seção 4.5). Como A3 e A4 diferem nos pesos e no *prompt* ao mesmo tempo, foi acrescentado o A3m, que usa o modelo e o *prompt* do A4 sem o adaptador. A diferença entre A4 e A3m mede só o ajuste. A diferença entre A4 e A3 mede a intervenção inteira: os pesos ajustados somados à dispensa da gramática.

O *prompt* indica o que o contexto informa sobre a notação. Há quatro *prompts*: um para XML, com um exemplo da notação; um para a DSL, com a gramática completa e um exemplo; a versão deste com a referência por rótulo (Seção 4.3); e o mínimo. Medidos com o tokenizador do Qwen, sem a descrição do processo, eles têm 1.147, 1.351, 1.452 e 753 *tokens*, respectivamente. As partes que descrevem o papel do modelo, o idioma, as regras de modelagem e o formato da resposta são idênticas em todos os *prompts*, e um teste automatizado garante

isso. Os exemplos do *prompt* de XML e do de DSL descrevem o mesmo processo. Assim, a comparação entre formatos não depende da redação da instrução.

Os demais braços são exploratórios, dado que foram definidos depois do congelamento e seus resultados são reportados à parte. O A2r foi definido depois de observados os resultados do A2, em que a maior parte das falhas sintáticas estava nas referências por identificador. Para voltar a um elemento já escrito, num laço ou numa convergência, o modelo precisava tê-lo marcado com um identificador quando o escreveu pela primeira vez, isto é, antecipar que ia precisar dele. O A2r repete o A2 com o mesmo modelo e o mesmo *prompt*, exceto no trecho sobre referências, onde passa a documentar a referência por rótulo, que aponta para trás sem declaração prévia (Seção 4.3). Na mesma ocasião, foram corrigidas duas imprecisões do *prompt* do A2 em relação à gramática, sobre onde cabem propriedades e como se escrevem identificadores. Como nenhuma saída do A2 usou as formas afetadas, essas correções não explicam a diferença entre os dois braços. Um teste automatizado garante que os dois *prompts* não diferem em mais nada. Como a gramática apenas ganhou uma forma alternativa, o modelo ajustado não precisou ser retreinado.

Os outros quatro braços exploratórios foram definidos antes de serem executados e respondem a duas perguntas sobre o A4. X-lc25 e X-lc50 repetem o treino do A4 com 25% e 50% dos pares de treino, selecionados por documento de origem e aninhados (os documentos de 25% estão contidos nos de 50%), para verificar se a qualidade ainda cresce com mais dados ou se já estabilizou; o ponto de 100% é o próprio A4. X-gr e X-mi repetem o treino com os mesmos dados e hiperparâmetros em modelos de outras famílias, o Granite 4.1 8B, da IBM, e o MiMo-7B-RL, da Xiaomi, para verificar se o resultado do A4 é do método ou do Qwen. Antes da execução, ficou registrado que todo braço treinado entraria no texto, qualquer que fosse o resultado. Um quinto braço, com o DeepSeek-Coder-6.7B, foi descartado sem que seus resultados fossem usados: o tokenizador desse modelo apaga os espaços do texto, de modo que o braço media o tokenizador, e não o método.

4.7 MEDIDAS

São quatro medidas: a validade, o custo por requisição, a razão de compressão e a fidelidade. As três primeiras são calculadas por programa, sem julgamento. A fidelidade exige julgamento e foi medida por um modelo de linguagem usado como juiz, cuja validação foi parcial (Seção 4.7.4). Por isso as conclusões principais do trabalho se apoiam na validade e no

custo, e a fidelidade é reportada como exploratória.

4.7.1 Validade

Uma saída é válida quando produz um BPMN 2.0 XML aceito pelo esquema XSD oficial. Nos braços que emitem XML, a saída é validada diretamente. Nos que emitem DSL, a saída precisa ser aceita pela gramática e ter todas as referências resolvidas, isto é, cada fluxo deve apontar para um elemento definido; só então é transpilada. Como o esquema XSD oficial aceitou todos os XMLs produzidos pelo transpilador, tanto no corpus quanto nas saídas dos braços (Seção 4.4), a validade desses braços equivale, na prática, à taxa de saídas que o transpilador consegue ler e converter. Saídas truncadas pelo limite de *tokens*, respostas com erro, saídas fora da gramática e saídas com referências a elementos inexistentes contam como inválidas. A validade de um braço é a proporção de saídas válidas entre todas as gerações: 46 itens com 3 repetições cada, ou 138 gerações (Seção 4.8).

A validade perante o esquema é uma condição necessária, e não suficiente: um XML válido pode descrever o processo errado. É por isso que a fidelidade é medida à parte.

4.7.2 Custo por requisição

O custo de uma requisição é

$$C = t_{\text{in}} \cdot p_{\text{in}} + t_{\text{out}} \cdot p_{\text{out}}, \tag{4.1}$$

em que t_{in} e t_{out} são as médias de *tokens* de entrada (*prompt* e descrição) e de saída das gerações de cada braço, e p_{in} e p_{out} são os preços por *token* do provedor (Tabela 4). Os *tokens* de todos os braços são contados com o mesmo tokenizador, o do Qwen2.5-Coder, o que é uma aproximação para os modelos de grande porte. Também é reportado o custo por saída válida, C dividido pela validade do braço, que incorpora o gasto com as gerações que não servem.

Todos os preços vêm de um único provedor, o Together AI, que oferece os três modelos, consultado em uma única data. Isso evita misturar tabelas de provedores diferentes e descontos por horário ou temporários. Dois cenários são calculados. No primeiro, toda a entrada é cobrada a preço cheio. No segundo, a parte do *prompt* anterior à descrição, que é igual em todas as requisições de um braço, é cobrada a preço de *cache*. Como o provedor não oferece *cache* para o modelo pequeno, o custo do A3 e do A4 é o mesmo nos dois cenários, o que desfavorece o A4 no segundo.

Tabela 4 – Preços usados no cálculo do custo (US$ por milhão de *tokens*)

Modelo	Entrada	Saída	Entrada em *cache*
DeepSeek V4 Pro	1,32	3,96	0,13
GLM 5.2	1,40	4,40	0,26
Qwen2.5 7B	0,30	0,30	—
Qwen2.5 7B com adaptador (hipótese)	0,45	0,45	—

Fonte**:** Elaborado pelo autor, com preços públicos do Together AI consultados em 24/10/2026 (Together AI, 2026b)

Nota**:** O preço do Qwen2.5 7B é o do Qwen2.5-7B-Instruct, que tem a mesma arquitetura e o mesmo porte do Qwen2.5-Coder-7B; nenhum provedor consultado oferecia a variante Coder por *token*. O acréscimo de 50% no cenário com adaptador é uma margem arbitrada, e não um preço observado. O provedor não oferece preço de *cache* para o Qwen2.5 7B, e toda a sua entrada é cobrada a preço cheio.

O preço do modelo base tem fonte: o Qwen2.5-7B-Instruct é oferecido a US$ 0,30 por milhão de *tokens*, e o único outro provedor encontrado cobrava menos (US$ 0,10 na entrada e US$ 0,20 na saída, no OpenRouter), de modo que o valor adotado é o menos favorável ao A4. O preço do adaptador, por outro lado, não existe no catálogo disponível dos provedores observados, por isso, foi estimado. Em outubro de 2026, os provedores consultados tinham encerrado a hospedagem de adaptadores LoRA cobrada por *token* e só os ofereciam em implantações dedicadas, cobradas por hora de uso (Together AI, 2026a; Fireworks AI, 2026). Por isso, o preço do A4 é uma hipótese, e não um valor observado. O cenário com acréscimo de 50% foi utilizado nas comparações, por ser o menos favorável ao A4. Para que a conclusão não dependa desse valor, também é calculado o preço de equilíbrio: o preço por *token* a partir do qual o A4 deixaria de ser mais barato que cada braço de grande porte, mantidos os *tokens* medidos e a validade de cada braço. O tempo de inferência não é medido, pois os modelos de grande porte rodaram na infraestrutura dos provedores e o modelo pequeno em uma estação local, de modo que a comparação mediria o equipamento, e não o método.

4.7.3 Razão de compressão

A razão de compressão de *tokens* (TCR) mede quanto a DSL é menor que o XML que ela produz:

$$\mathrm{TCR} = \frac{\mathrm{tokens}(\mathrm{XML})}{\mathrm{tokens}(\mathrm{DSL})}, \tag{4.2}$$

com o tokenizador do Qwen2.5-Coder. O XML do numerador é o produzido pelo transpilador, sem as coordenadas do leiaute, que inflariam a razão sem acrescentar conteúdo ao processo

(Seção 4.4). A TCR é calculada sobre o corpus e sobre as saídas dos braços que emitem DSL, considerando apenas as saídas que passam pela transpilação. Nos braços que emitem XML não há representação intermediária, e a razão não é definida.

4.7.4 Fidelidade

A fidelidade é o quanto o modelo gerado representa o processo descrito no texto. Neste trabalho, ser fiel é representar o que o texto diz sem inventar conteúdo: um modelo que acrescenta atividades ou decisões que o texto não sustenta é penalizado, mesmo que o acréscimo seja plausível. Essa definição foi escolhida porque, nas saídas de um modelo de linguagem, inventar conteúdo é justamente o erro que importa detectar.

4.7.4.1 O juiz

A fidelidade é medida por um modelo de linguagem usado como juiz, o `muse-spark-1.3-contributor`, da Meta. Dois critérios orientaram a escolha. O primeiro é a família: o juiz é de família diferente da de todos os modelos que geraram o corpus ou participam dos braços, o que reduz o risco de ele favorecer saídas parecidas com as que ele mesmo produziria. O segundo é a capacidade: no índice de inteligência do Artificial Analysis, que agrega avaliações independentes de modelos de linguagem (Artificial Analysis, 2026), o juiz obtém entre 45 e 48 pontos, conforme a configuração de raciocínio, contra no máximo 36 dos modelos de grande porte que geraram o corpus ou participam dos braços, cada um na sua configuração mais alta. Capacidade geral, porém, não garante que o juiz avalie fidelidade como um especialista, e por isso ele foi validado de duas formas antes de ser usado nos braços, como descrito a seguir.

O juiz recebe o texto original do processo e o modelo gerado, e devolve uma nota inteira de 0 a 5.

O juiz não vê o XML. O modelo é apresentado como uma lista neutra de elementos e fluxos, com tipo e rótulo de cada elemento e identificadores sem significado (N1, N2, . . .). Isso remove as diferenças de forma entre um XML escrito diretamente por um modelo e um XML produzido pelo transpilador, de modo que o juiz não sabe de que braço veio a saída.

A rubrica do juiz tem cinco critérios: as atividades do texto estão presentes e nenhuma foi inventada; as decisões aparecem com as saídas que o texto dá; o paralelismo corresponde ao texto; a ordem, as repetições e os finais correspondem ao texto; e o modelo é bem formado, com início, fim e todos os elementos alcançáveis. Cada nota de 0 a 5 tem uma descrição na rubrica.

Antes de dar a nota, o juiz lista os problemas encontrados em cada critério, em uma resposta com formato fixo, o que permite auditar cada nota. Quem executa cada atividade não é avaliado.

Cada saída é julgada três vezes, com temperatura zero e sementes diferentes, e a nota da saída é a mediana das três. O texto completo das instruções do juiz é fixado por um *hash*, gravado com cada julgamento, de modo que uma mudança na rubrica gera um novo conjunto de notas, e não uma mistura de versões. As instruções estão no Apêndice B.

4.7.4.2 Validação contra notas de especialista

A primeira validação comparou as notas do juiz com as do especialista no Zenodo: 222 modelos de 24 processos, cada um com uma nota de 0 a 5 (MANGLER; KLIEVTSOVA, 2023). O critério, fixado antes da execução, era uma correlação de Spearman de pelo menos 0,5 entre as duas notas, com intervalo de confiança obtido por *bootstrap* sobre os processos.

O juiz não passou. Na primeira tentativa, a escala tinha passos de meio ponto, e o juiz respondeu quase só com valores terminados em 0,5, o que reduziu a escala a cinco níveis deslocados; a correlação foi de 0,133, com intervalo de [0,005; 0,263]. Na segunda tentativa, com a escala de inteiros e a rubrica inalterada, o juiz passou a usar todos os níveis, e a correlação foi de 0,108 [−0,028; 0,258].

A leitura dos 10 casos de maior divergência, feita pelo autor, indicou que o juiz e o especialista não medem a mesma coisa. Em todos os 10, os problemas que o juiz apontou existiam de fato no modelo; o desacordo estava no peso dado a eles. A principal causa é a definição de fidelidade: os textos do Zenodo são enunciados de projeto vagos, como “crie um processo que dê suporte a...”, em que quem modela precisa inventar detalhes, e o especialista premia uma boa invenção, enquanto a rubrica do juiz a penaliza. Somam-se a isso três fatores: as notas vêm de um único avaliador, 48% delas são a nota máxima, e 8 dos 222 modelos têm fluxos que, no XML, não estão ligados aos elementos, embora pareçam ligados no desenho que o especialista avaliou. Como o juiz está entre os modelos mais bem posicionados no índice citado acima, a falta de concordância dificilmente se explica por falta de capacidade: ela aponta, antes, para a diferença entre o que a rubrica e o especialista consideram fiel.

A rubrica não foi ajustada para se aproximar do especialista. Uma terceira tentativa, feita depois de ver dois resultados, seria um ajuste aos dados, e uma rubrica que não penaliza acréscimos poderia premiar exatamente as invenções que se quer detectar nos braços.

4.7.4.3 Validação por perturbação controlada

A segunda validação usa casos em que a resposta certa é conhecida por construção. Partindo do modelo de referência de cada um dos 46 itens do PMo, foram gerados mutantes com defeitos conhecidos: a remoção de uma atividade, a inserção de uma atividade inventada, a troca dos rótulos de duas atividades, a troca de um paralelismo por uma escolha exclusiva e a troca dos destinos de dois ramos de uma decisão. Cada item recebeu ainda um mutante com três defeitos sorteados e uma paráfrase, em que todos os rótulos foram reescritos com o mesmo sentido por outro modelo de linguagem, o GLM 5.3. A reescrita foi leve, em geral trocando só o verbo: a semelhança média entre o rótulo original e a paráfrase foi de 0,41 pelo índice de Jaccard, o que limita o quanto a invariância (C3) demonstra. Ao todo, foram 343 casos.

Os critérios foram fixados antes de qualquer julgamento e comparam cada caso com o modelo intacto do mesmo item. A sensibilidade (C1) é a proporção de pares em que o intacto recebe nota maior que o mutante com um defeito, com empates contando meio ponto, e deve ser de pelo menos 0,80. A gradação (C2) é a correlação de Spearman entre o número de defeitos (0, 1 ou 3) e a variação da nota em relação ao intacto, e deve ser de no máximo $-0{,}5$. A invariância (C3) é a proporção de itens em que a paráfrase muda a nota em no máximo 20% da escala, e deve ser de pelo menos 0,80. A medida originalmente prevista para a fidelidade, o DF-F1 (Seção 4.7.5), recebeu o mesmo cálculo, como comparação. A Tabela 5 mostra os resultados.

Tabela 5 – Validação por perturbação controlada

Critério	Limiar	Juiz	DF-F1
C1 sensibilidade	$\geq 0{,}80$	0,702	0,837
C2 gradação	$\leq -0{,}5$	$-0{,}385$	$-0{,}588$
C3 invariância	$\geq 0{,}80$	1,000	0,000

Fonte**:** Elaborado pelo autor

Nenhuma das duas medidas passou. O DF-F1 falhou na invariância: a paráfrase derrubou sua nota, em média, em 0,80 numa escala de 0 a 1, embora o processo fosse o mesmo. Ele também não percebe a troca de paralelismo por escolha nem a troca de ramos, com concordância de 0,5 nos dois casos, porque a nota não muda. O juiz foi perfeito na invariância, mas ficou abaixo do limiar na sensibilidade e na gradação.

A análise dos erros do juiz, feita depois do resultado, mostrou que ele quase nunca inverte a ordem: em 205 pares de intacto e mutante com um defeito, o mutante recebeu nota maior em apenas 10. O que derruba a sensibilidade são os empates, que vêm de um efeito de

piso: em 28 dos 46 itens, o próprio modelo de referência recebeu nota 3 ou menor, e um defeito a mais em um modelo que já tem problemas nem sempre muda a nota. A concordância cresce com a nota do intacto, de 0,47 quando ela é 2 a 0,97 quando é 5. Esse piso tem uma causa identificada: em pelo menos 10 dos itens compartilhados com o Zenodo, a referência do PMo é um modelo feito por iniciante (Seção 4.2), e as críticas do juiz a essas referências procedem nos casos lidos. Nos 22 itens que não vêm do Zenodo, o juiz teria passado em C1 (0,826) e C2 ($-0{,}536$). Esse recorte, porém, foi feito depois de ver os dados e não substitui o critério fixado antes.

4.7.4.4 Uso da medida

Nenhuma medida de fidelidade passou nas validações fixadas antes da execução. O juiz tem evidências parciais a seu favor: é insensível à paráfrase, quase não inverte a ordem entre o modelo correto e o defeituoso, e passaria nos critérios nos itens cuja referência não vem do Zenodo, embora esse recorte seja posterior ao resultado. Por isso ele é usado, mas a fidelidade é reportada como medida exploratória, e nenhuma conclusão principal depende dela.

O juiz foi aplicado às saídas válidas dos braços A1, A1g, A2, A2g, A2r e A4. Uma saída inválida não tem modelo a julgar. A nota de um braço em um item é a média das notas das suas saídas válidas, e cada braço é reportado com a sua cobertura, isto é, o número de itens em que teve ao menos uma saída válida. Como os 24 itens de origem do Zenodo têm textos vagos e se comportaram de forma diferente na validação, a fidelidade também é reportada separadamente para eles e para os 22 demais.

4.7.5 A medida originalmente prevista: DF-F1

O protocolo inicial previa como medida de fidelidade o DF-F1, que compara a saída com o modelo de referência pelos pares de atividades e eventos que se sucedem no fluxo, após alinhar os rótulos, e resume a concordância por um F1. A medida tem duas falhas que só foram verificadas depois do congelamento do protocolo. Ela não concorda com o especialista do Zenodo (correlação de Spearman de 0,04) e falhou na validação por perturbação, como mostra a Tabela 5. Além disso, depende do modelo de referência, que em parte dos itens não é confiável. Por isso foi substituída pelo juiz, o que é declarado como desvio do protocolo (Seção 4.9). Os resultados das hipóteses originais, decididas pelo DF-F1, são apresentados no Apêndice D, apenas para registro.

4.8 PROTOCOLO E ANÁLISE ESTATÍSTICA

4.8.1 Execução

Cada braço gerou uma saída para cada um dos 46 itens do conjunto de teste, três vezes, num total de 138 gerações por braço. Os braços de grande porte foram chamados com temperatura zero, reduzindo assim a variação entre as repetições, mas não a eliminando, pois o mesmo pedido pode produzir saídas diferentes no provedor. Os braços do modelo pequeno rodaram localmente com decodificação gulosa, em que a repetição tende a ser idêntica. As três repetições foram mantidas em todos os braços, para que a variação entre elas seja medida, e não suposta.

O limite de *tokens* de saída foi de 8.192 nos braços que emitem XML e de 2.048 nos que emitem DSL. A diferença acompanha o tamanho de cada formato, já que o XML é várias vezes mais longo que a DSL equivalente (Seção 4.7.3). Uma saída que atinge o limite é registrada como truncada e conta como inválida (Seção 4.7.1). Com esses limites, foram truncadas 3 gerações do A1, 2 do A1g e 81 do A3m, e nenhuma dos demais braços. Nenhuma geração foi repetida em caso de falha, já que uma falha é parte do resultado do braço. Cada geração foi gravada com o modelo, o *hash* do *prompt* e a versão do código que a produziu.

4.8.2 Unidade de análise e comparações

A unidade de análise é o item. Como todos os braços recebem os mesmos 46 itens, as comparações entre dois braços são pareadas: calcula-se a diferença item a item, e o resultado é a média dessas diferenças, com intervalo de confiança de 95% obtido por *bootstrap* sobre os itens, com 10.000 reamostragens e semente fixa. O *bootstrap* foi escolhido por não exigir que as diferenças sigam uma distribuição conhecida, o que não vale para proporções de três repetições nem para notas inteiras.

Na validade, o valor de um braço em um item é a fração das três repetições que são válidas, isto é, 0, 1/3, 2/3 ou 1. Na fidelidade, também é reportado o teste de Wilcoxon pareado, que indica se as diferenças entre os braços pendem para um dos lados mais do que o acaso explicaria. O teste foi escolhido por não supor que as diferenças sigam uma distribuição normal, o que não vale para notas inteiras de 0 a 5, e é bilateral, isto é, considera tanto a possibilidade de um braço ser melhor quanto a de ser pior.

4.8.3 Equivalência

A ausência de diferença significativa não demonstra que dois braços são equivalentes: com 46 itens, um intervalo largo pode conter tanto o zero quanto diferenças grandes. Por isso, toda afirmação de empate exige uma margem de equivalência fixada de antemão. A regra é a mesma para as duas medidas. Se o intervalo de 95% da diferença cai inteiro dentro da margem, os braços são declarados equivalentes. Se cai inteiro fora, são declarados diferentes além da margem. Nos demais casos, o resultado é inconclusivo.

Na validade, a margem é de ± 10 pontos percentuais, fixada antes do cálculo dos intervalos, quando apenas as proporções de cada braço tinham sido vistas. Uma diferença menor que essa não altera a ordem de custo por saída válida entre os braços. Os contrastes são A4 $-$ A2r, que compara o modelo pequeno com o método executado pelo modelo de grande porte, e A2r $-$ A1, que compara o método com a geração direta de XML.

Na fidelidade, a margem é de $\pm 0{,}5$ ponto na escala de 0 a 5, e foi fixada depois de vistas as notas dos braços, o que torna esses vereditos exploratórios. A margem se justifica sem recorrer a essas notas: é metade do passo da escala inteira e é menor que a queda média de nota causada por um único defeito na validação por perturbação, de 0,63 ponto. Os contrastes são A2r $-$ A1, A2 $-$ A1, A4 $-$ A2r e A4 $-$ A1.

4.8.4 Custo e compressão

O custo e a razão de compressão são descritivos: reportam-se os valores de cada braço, sem teste. A conclusão sobre o custo é verificada pelo preço de equilíbrio do A4 (Seção 4.7.2), que diz quanto o preço do modelo pequeno poderia subir antes de a ordem entre os braços se inverter.

4.8.5 Estatuto dos resultados

O protocolo inicial, congelado antes da primeira execução, previa quatro hipóteses, três delas decididas pelo DF-F1, com teste de Wilcoxon pareado bilateral sobre a mediana das três repetições por item, correção de Holm sobre os três contrastes planejados (A2 $-$ A1, A2g $-$ A1g e A4 $-$ A2) e intervalos por *bootstrap*. Como o DF-F1 foi substituído (Seção 4.7.5), essa análise é reportada integralmente, mas apenas no Apêndice D.

As análises do corpo do trabalho foram definidas depois do congelamento, e o texto

as distingue pelo momento em que cada decisão foi tomada. A margem da validade foi fixada antes do cálculo dos intervalos. A margem da fidelidade, depois de vistas as notas. A fidelidade inteira é exploratória, pela validação parcial do juiz, e não recebe correção para comparações múltiplas. O braço A2r foi definido depois de vistos os resultados do A2. Os desvios em relação ao protocolo inicial estão reunidos na Seção 4.9.

4.9 DESVIOS DO PROTOCOLO INICIAL

O protocolo inicial fixou, antes da primeira execução, os braços, os *prompts*, as medidas, as hipóteses e a análise estatística. Ele admitia mudanças, desde que declaradas com a data, o motivo e a indicação de terem sido feitas antes ou depois de vistos os resultados. Uma mudança feita depois dos resultados é mais fraca, porque pode ter sido escolhida por favorecer as hipóteses do autor. Por isso, para cada desvio, é indicado também em que direção ele poderia mover os resultados. A Tabela 6 resume os desvios, descritos a seguir.

Tabela 6 – Desvios do protocolo inicial

Desvio	Data	Resultados	Efeito possível
Glosa no *prompt* de DSL	16/08/2026	depois (A3)	eleva o A3, contra a hipótese
Correção de identificadores duplicados	25/08/2026	depois	eleva a DSL, a favor
Braço A2r	12/10/2026	depois	a favor; braço exploratório
Medida de custo	12/10/2026	depois	medida nova
Exclusão de 7 itens vazados	14/10/2026	depois	nenhum previsível
Correções P9 e P10 do transpilador	18/10/2026	depois	reduz a TCR, contra
Substituição do DF-F1 pelo juiz	18–20/10/2026	depois	medida nova
Erro no desenho da perturbação	20/10/2026	depois	recorte posterior
Margens de equivalência	20/10/2026	ver texto	análise nova

Fonte**:** Elaborado pelo autor

Nota**:** A coluna “Resultados” indica se o desvio foi decidido antes ou depois de vistos os resultados que ele afeta.

4.9.1 Mudanças nos artefatos

Glosa no *prompt* de DSL. Depois da execução do A3, verificou-se que o *prompt* de DSL listava as palavras-chave dos *gateways* sem ligá-las aos conceitos de BPMN usados nas regras de modelagem, e mostrava duas sintaxes de ramificação sem avisar que uma não se generaliza para a outra. O *prompt* de XML não precisa dessa ligação, porque a terminologia

BPMN já está no pré-treinamento dos modelos, e a omissão prejudicava apenas os braços de DSL. Os dois defeitos são verificáveis lendo o *prompt* e testando construções no *parser*, sem recorrer a nenhum resultado. O *prompt* foi corrigido e o A3 foi executado de novo; o A2 e o A2g já foram executados com a versão corrigida. A mudança é posterior aos resultados do A3, mas só podia elevar o A3, o que reduz a vantagem que o A4 poderia mostrar sobre ele. O efeito foi pequeno, e as gerações anteriores foram preservadas.

Correção de identificadores duplicados. Quando uma DSL redeclarava um elemento já marcado com identificador, o transpilador emitia dois elementos com o mesmo identificador, e o XML era rejeitado pelo esquema. A correção é posterior aos resultados e só podia elevar a validade dos braços de DSL, isto é, favorecer o método. Por isso, foi aplicada a todos os braços ao mesmo tempo, retranspilando as saídas já gravadas, sem nova chamada a modelo. Mudaram 6 de 1.590 gerações, nenhuma do A4, e nenhum desfecho de hipótese mudou.

Correções P9 e P10 do transpilador. Antes da execução do juiz, foram corrigidos dois defeitos. O primeiro (P9) ligava ao *gateway* de convergência um ramo que terminava em evento de fim, gerando um fluxo saindo do fim. O segundo (P10) ignorava a raia indicada no próprio elemento e repetia a mesma raia, com todos os membros, quando o bloco aparecia mais de uma vez. Nenhuma das correções alterou a validade de nenhum braço. Todas as saídas e o corpus foram retranspilados, e o corpus continuou inteiramente válido. A P10 afetou a razão de compressão: as raias repetidas inflavam o XML, e a TCR do corpus caiu de 6,01 para 5,37 (Seção 4.7.3). O efeito é contrário à hipótese do trabalho, e os valores reportados são os corrigidos.

Braço A2r. O A2r foi definido depois de vistos os resultados do A2, a partir da análise das falhas de sintaxe (Seção 4.6). É, portanto, exploratório, e seus resultados são reportados como tais, mesmo sendo o braço do método com maior validade.

4.9.2 Mudanças nos dados

Exclusão de 7 itens. Uma auditoria posterior aos resultados encontrou 7 itens do PMo cujos textos também estão no PET, usado no treino (Seção 4.2.3). Os 7 itens foram excluídos de todas as análises, que passaram de 53 para 46 itens, sem apagar nenhuma geração. A exclusão segue um critério definido sem olhar o desempenho dos braços, a sobreposição de texto com o treino, e foi aplicada a todos os braços. As validades mudaram no máximo 3,7 pontos percentuais, e a ordem entre os braços se manteve.

4.9.3 Mudanças nas medidas e na análise

Medida de custo. O custo por requisição não estava no protocolo inicial. Ele não é uma medição nova, e sim a conversão em dinheiro de quantidades já medidas, os *tokens* de entrada e de saída de cada geração. Os preços e as hipóteses do cálculo estão na Seção 4.7.2.

Substituição do DF-F1 pelo juiz. A medida de fidelidade prevista, o DF-F1, falhou nas duas validações feitas depois do congelamento, e foi substituída pelo juiz (Seções 4.7.4 e 4.7.5). O juiz também não passou nos critérios fixados, e por isso a fidelidade é exploratória. As hipóteses originais, decididas pelo DF-F1, são reportadas no Apêndice D. Na validação contra o Zenodo, o juiz foi executado duas vezes: a segunda corrigiu a escala de meio ponto, que o juiz não usava por inteiro, e manteve a rubrica. As duas tentativas são reportadas.

Erro no desenho da perturbação. A validação por perturbação supunha que o modelo de referência de cada item estivesse correto. Depois do resultado, verificou-se que ao menos 10 referências são modelos de iniciantes do Zenodo (Seção 4.2), o que já se sabia antes do teste e deveria ter excluído esses itens da base. O teste mediu, em parte, a qualidade da referência, e não só a do juiz. O resultado nos 22 itens fora do Zenodo é reportado, mas como recorte posterior ao resultado, sem valor de validação.

Margens de equivalência e unidade da validade. O protocolo inicial não previa margens de equivalência, e um empate só podia ser descrito como ausência de diferença significativa. As margens foram acrescentadas depois do congelamento (Seção 4.8). A da validade foi fixada antes do cálculo dos intervalos, e a da fidelidade, depois de vistas as notas. Na mesma ocasião, a validade de um item passou a ser a fração das três repetições válidas, e não a mediana prevista no protocolo inicial. Com três repetições que valem 0 ou 1, a mediana vira um voto de maioria e descarta a informação de que, por exemplo, uma de três saídas falhou.

4.10 AMEAÇAS À VALIDADE

As ameaças estão agrupadas pelo tipo de conclusão que afetam, se os efeitos observados se devem aos braços (validade interna), se as medidas medem o que se pretende (validade de construto), se os resultados se estendem além deste estudo (validade externa) e se a análise sustenta as conclusões tiradas (validade da conclusão).

4.10.1 Validade interna

Decisões tomadas depois dos resultados. Parte das decisões foi tomada depois de vistos os resultados, o braço A2r, a medida de custo, a substituição do DF-F1 e a margem de equivalência da fidelidade (Seção 4.9). Cada uma pode ter sido influenciada, ainda que sem intenção, pelo que favorecia o trabalho. A mitigação adotada foi declarar o momento de cada decisão, reportar a análise original no apêndice e aplicar toda correção a todos os braços ao mesmo tempo.

Redação dos *prompts*. Cada formato foi testado com um único *prompt*, e testar com outras variantes poderiam mudar os resultados. As partes comuns são idênticas entre os *prompts*, o que torna a comparação entre formatos justa, mas não garante que cada *prompt* seja o melhor possível para o seu formato. A emenda da glosa (Seção 4.9) mostrou que uma correção de redação teve efeito pequeno, mas uma única emenda não basta para descartar essa sensibilidade.

Variação entre execuções e versões dos modelos. A temperatura zero não garante saídas idênticas nos provedores, e os modelos de grande porte foram acessados por API, cujas versões podem mudar sem aviso. As três repetições medem a variação dentro do período de execução, mas uma nova execução no futuro pode não reproduzir exatamente os números. Os resultados do modelo pequeno, executado localmente com pesos fixos, não têm essa limitação.

4.10.2 Validade de construto

Fidelidade. O juiz não passou nos critérios de validação fixados (Seção 4.7.4), e por isso a fidelidade é exploratória. Mesmo como medida exploratória, a escala do juiz não é calibrada, pois uma diferença de um ponto não corresponde a um número conhecido de defeitos, e as notas de braços diferentes só podem ser comparadas entre si, e não interpretadas em termos absolutos.

Referências e textos de avaliação. Ao menos 10 das referências do PMo são modelos feitos por iniciantes, e os 24 itens vindos do Zenodo têm enunciados de projeto vagos, em que quem modela precisa inventar parte do processo (Seção 4.2). Isso afeta qualquer medida que dependa da referência ou que penalize acréscimos. A fidelidade é julgada contra o texto, e não contra a referência, e é reportada separadamente para os dois grupos de itens, mas a vagueza desses textos continua afetando o julgamento.

Validade. A validade mede apenas a conformidade com o esquema XSD. Um XML válido pode ter elementos inalcançáveis, decisões sem saída ou um processo diferente do descrito,

e esses problemas só são capturados, em parte, pela fidelidade, que é exploratória.

Custo. O preço do modelo com adaptador é uma hipótese, porque nenhum provedor consultado oferecia esse serviço cobrando por *token*, e todos os preços são de um único provedor em uma única data. Além disso, os *tokens* de todos os braços foram contados com o tokenizador do Qwen, o que é uma aproximação para os modelos de grande porte. O preço de equilíbrio (Seção 4.7.2) reduz a dependência da conclusão em relação ao preço exato, mas não a elimina.

4.10.3 Validade externa

Domínio dos dados de treino. 94,4% dos pares de treino vêm do *handbook* da GitLab, de uma única empresa, enquanto o teste é feito sobre processos de outros domínios. Isso pode subestimar o modelo pequeno, que foi avaliado fora do domínio em que foi treinado, mas também significa que não se sabe como ele se comportaria com dados de treino mais variados. O desenho não separa o efeito dessa mudança de domínio do efeito do tamanho do modelo.

Dados de treino gerados por modelo. Os pares de treino foram gerados pelo GLM 5.2 e filtrados apenas pela consistência entre o JSON e o XML, sem verificação humana de que o JSON representa o texto. Erros do gerador podem ter sido aprendidos pelo modelo pequeno. O GLM 5.2 também é usado nos braços A1g e A2g, o que pode favorecê-lo nesses braços, já que o corpus segue o seu estilo de modelagem; por isso a comparação entre formatos foi replicada com o DeepSeek V4 Pro, que não participou da geração.

Contaminação dos modelos de grande porte. O PMo, o Zenodo e o PET são conjuntos públicos, e é possível que seus textos e modelos de referência estejam nos dados de pré-treinamento dos modelos de grande porte. Se estiverem, os braços A1, A1g, A2, A2g e A2r são favorecidos, e a comparação com o modelo pequeno fica desfavorável a ele. Não há como verificar essa contaminação em modelos de pesos fechados ou de dados de treino não publicados.

Alcance do estudo. O estudo usa uma linguagem de saída (BPMN 2.0), uma língua (inglês), uma família de modelo pequeno no braço principal e 46 itens de teste. Os braços com o Granite e o MiMo verificam parte da dependência em relação ao modelo pequeno, mas a generalização para outras linguagens XML, que é a motivação do método, não foi testada.

4.10.4 Validade da conclusão

Tamanho da amostra. Com 46 itens, os intervalos de confiança das diferenças de validade têm cerca de ± 13 pontos percentuais, mais largos que a margem de equivalência de 10

pontos. Por isso, a igualdade de validade entre o A4 e o A2r não pode ser declarada equivalência, e o estudo não tem itens suficientes para isso (Seção 4.8).

Comparações múltiplas. As análises de fidelidade e os braços exploratórios não recebem correção para comparações múltiplas. Com vários contrastes, a chance de algum resultado aparentemente significativo ser fruto do acaso aumenta, e esses resultados devem ser lidos como indicações, e não como confirmações.

5 RESULTADOS

5.1 VISÃO GERAL

Este capítulo segue a ordem das medidas da Seção 4.7. Primeiro vêm a validade (Seção 5.2), o custo (Seção 5.3) e a compressão (Seção 5.4) que sustentam as conclusões principais. Depois é mostrada a fidelidade, que é exploratória (Seção 5.5), e uma análise do modelo especializado, que reúne os braços do modelo pequeno e os braços exploratórios (Seção 5.6). A Tabela 7 apresenta de forma resumida os braços principais nas três medidas que os comparam diretamente.

Tabela 7 – Panorama dos braços

Braço	Modelo	Saída	Validade	Itens cobertos	Fidelidade	Custo
A1	DeepSeek V4 Pro	XML	97,8%	46	3,77	5.561
A1g	GLM 5.2	XML	93,5%	46	3,32	6.386
A2	DeepSeek V4 Pro	DSL	62,3%	34	3,67	4.311
A2g	GLM 5.2	DSL	52,9%	35	3,49	5.505
A3	Qwen 7B	DSL	58,7%	27	—	874
A3m	Qwen 7B	DSL	0,0%	0	—	—
A4	Qwen 7B†	DSL	87,0%	40	2,78	591
A2r	DeepSeek V4 Pro	DSL	87,0%	43	3,85	3.241

Fonte**:** Elaborado pelo autor

Nota**:** Validade: proporção das 138 gerações (46 itens, 3 repetições) aceitas pelo esquema XSD. Itens cobertos: itens com ao menos uma saída válida, de 46. Fidelidade: média das notas do juiz de 0 a 5 nos itens cobertos, medida exploratória (Seção 4.7.4); o juiz não foi aplicado ao A3 e ao A3m. Custo: US$ por milhão de saídas válidas, sem *cache*; para o A4, no cenário com acréscimo de 50% pelo adaptador. † com o adaptador treinado. O A2r é exploratório.

A geração direta de XML teve a maior validade: 97,8% no A1 e 93,5% no A1g. Com os mesmos modelos, pedir a DSL no lugar do XML reduziu a validade para 62,3% e 52,9%, o contrário do que o protocolo inicial previa. A maioria dessas falhas estava nas referências a elementos já escritos, e o A2r, que acrescenta a referência por rótulo, elevou a validade do mesmo modelo para 87,0%. Mesmo assim, ele continua abaixo da geração direta (Seção 5.2).

O modelo pequeno especializado, o A4, teve a mesma validade do A2r, 87,0%, sem a gramática no *prompt*. A igualdade, porém, é uma coincidência: dos itens em que algum dos dois falhou, só um é comum aos dois braços, de modo que eles não esbarram em uma mesma limitação da notação (Seção 5.2). Ela também não permite afirmar que os dois são equivalentes, porque com 46 itens a incerteza da diferença é maior que a margem fixada (Seção 4.8). O custo

por saída válida do A4 é de US$ 591 por milhão, contra US$ 3.241 do A2r e US$ 5.561 do A1, mesmo no cenário de preço mais desfavorável ao A4 (Seção 5.3).

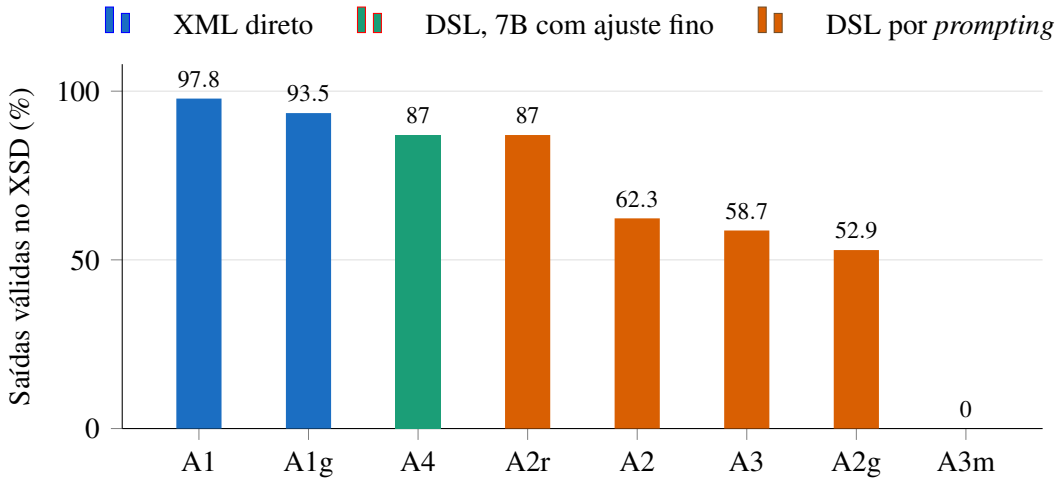
Na fidelidade, que é exploratória, o A2r e o A1 ficaram no mesmo nível, e o A4 ficou cerca de um ponto abaixo de ambos, uma diferença maior que a margem fixada (Seção 5.5). O resultado do A4 é, portanto, uma troca: a mesma validade do método no modelo de grande porte, a um custo várias vezes menor, com menor fidelidade.

5.2 VALIDADE

5.2.1 Validade por braço

A Figura 2 mostra a validade de cada braço, isto é, a proporção das 138 gerações que produziram um XML aceito pelo esquema. Os braços se dividem em três grupos. A geração direta de XML ficou acima de 90%, com 97,8% no A1 e 93,5% no A1g. O A4 e o A2r ficaram em 87,0%. Os braços que pedem a DSL pela gramática no *prompt*, sem a referência por rótulo, ficaram entre 52,9% e 62,3%: o A2, o A2g e o A3.

Figura 2 – Validade por braço experimental



Fonte**:** Elaborado pelo autor

Nota**:** Percentual sobre as 138 gerações de cada braço (46 itens, 3 repetições). O A2r é exploratório.

O A3m não produziu nenhuma saída válida, e o motivo não é a dificuldade da notação. Sem o adaptador e sem a gramática no *prompt*, o modelo não tem como saber o que é a DSL, e escreveu BPMN XML nas 138 gerações, o formato que conhece do pré-treinamento. Em 81 delas, o XML passou do limite de 2.048 *tokens*, dimensionado para a DSL, e foi truncado. O zero do A3m mostra que o *prompt* mínimo, sozinho, não transmite a notação; é com esse mesmo

prompt que o A4 chega a 87,0%, e a diferença entre os dois é apenas o adaptador.

Entre os três braços do modelo pequeno, a validade passa de 0% sem a notação, a 58,7% com a gramática no *prompt* (A3) e a 87,0% com a notação aprendida no ajuste (A4). O A4 ainda usa um *prompt* 598 *tokens* menor que o do A3, porque dispensa a gramática (Seção 4.6).

5.2.2 Comparações com margem de equivalência

A Tabela 8 mostra os dois contrastes de validade definidos na Seção 4.8, com a margem de ± 10 pontos percentuais.

Tabela 8 – Contrastes de validade

Contraste	Diferença	IC 95%	Veredito
A4 − A2r	0,0	[−13,8; 13,0]	inconclusivo
A2r − A1	−10,9	[−21,0; −1,4]	inconclusivo

Fonte**:** Elaborado pelo autor

Nota**:** Diferença média da validade por item, em pontos percentuais, pareada nos 46 itens. Intervalo de confiança por *bootstrap* com 10.000 reamostragens. Margem de equivalência de ± 10 pontos, fixada antes do cálculo dos intervalos.

Os dois contrastes são inconclusivos, mas por motivos diferentes. Entre o A4 e o A2r não há diferença observada, mas o intervalo vai de $-13{,}8$ a 13,0 pontos e ultrapassa a margem dos dois lados, de modo que não é possível afirmar que os dois são equivalentes. Entre o A2r e o A1, o intervalo inteiro está abaixo de zero: o A2r é menos válido que a geração direta de XML. A perda pode, porém, ser menor que 10 pontos, porque o intervalo começa em $-21{,}0$ e termina em $-1{,}4$, e por isso também não é possível afirmar que ela ultrapassa a margem.

5.2.3 Causas de invalidade

A Tabela 9 classifica as saídas inválidas dos braços de DSL pela causa da rejeição.

No A2 e no A2g, a referência é a principal causa de falha: 38 das 52 saídas inválidas do A2 e 41 das 65 do A2g. Em parte delas, o modelo referencia um identificador que nunca declarou. Nas demais, a marca de referência aparece onde a gramática não a aceita, em geral numa tentativa de declarar o identificador depois de já tê-lo usado. Nos dois casos, o problema é o mesmo: para voltar a um elemento, o modelo precisava tê-lo marcado com um identificador quando o escreveu pela primeira vez, antes de saber que voltaria a ele. Foi esse diagnóstico

Tabela 9 – Causas de invalidade nos braços de DSL

Braço	Inválidas	Referência	Outra sintaxe	Bloco aberto	Truncada
A2	52	38	14	0	0
A2g	65	41	23	1	0
A2r	18	8	10	0	0
A3	57	0	57	0	0
A3m	138	0	57	0	81
A4	18	12	0	6	0

Fonte**:** Elaborado pelo autor

Nota**:** Contagem sobre as 138 gerações de cada braço. Referência: referência a um identificador nunca declarado, ou marca de referência (#) em posição que a gramática não aceita. Bloco aberto: a saída termina com um bloco sem fechamento. No A3m, todas as saídas são XML, e não DSL.

que motivou o A2r (Seção 4.6), e com a referência por rótulo as falhas de referência do mesmo modelo caíram de 38 para 8. As 8 que restaram são referências a identificadores nunca declarados, em 4 itens.

O A4 e o A2r têm 18 saídas inválidas cada, mas falham de formas diferentes. O A4 falha em 6 itens, sempre nas três repetições, o que é esperado da decodificação gulosa. Suas falhas são 12 referências a identificadores nunca declarados e 6 blocos não fechados. A referência continua sendo o principal problema do A4, o que é coerente com o treino, que só contém a referência por identificador (Seção 4.4). O A2r falha em 8 itens, a maioria em apenas uma ou duas das três repetições. Só um item, o pmo_32, falha nos dois braços. Se os dois esbarrassem em uma construção que a DSL não consegue expressar, as falhas cairiam nos mesmos itens.

O A3 não tem nenhuma falha de referência. Suas 57 saídas inválidas são erros de sintaxe de outra natureza, como palavras-chave inventadas e construções fora da gramática, o que indica que o modelo pequeno sem ajuste não domina a notação nem com a gramática no *prompt*.

5.3 CUSTO

5.4 COMPRESSÃO

5.5 FIDELIDADE (EXPLORATÓRIA)

5.6 O MODELO ESPECIALIZADO

5.7 SÍNTESE

6 CONCLUSÕES E TRABALHOS FUTUROS

O objetivo geral deste trabalho (Seção 1.2.1) foi investigar se a geração de uma representação intermediária comprimida, seguida de expansão determinística, produz modelos BPMN 2.0 válidos de forma mais confiável e mais econômica em *tokens* do que a geração direta de XML. A resposta obtida é **parcialmente afirmativa, e sua parte negativa é tão informativa quanto a positiva**.

Quanto à economia, a resposta é inequívoca: a representação proposta reduz em cerca de 85% o volume de *tokens* que o modelo precisa emitir, com razão de compressão em torno de 7 medida sobre as saídas efetivamente geradas. Quanto à confiabilidade, a resposta dependeu menos da novidade da notação do que de um traço de seu desenho. Na primeira versão, fornecida por *prompt*, a notação **piorou** o resultado: os braços que descreviam a gramática no contexto, com os modelos DeepSeek e GLM, obtiveram 64,8% e 56,6% de validade, respectivamente, contra 98,1% e 94,3% da geração direta de XML pelos mesmos modelos. A análise das falhas localizou a causa na forma de referência, que exigia declarar identificadores antes de saber que seriam usados; acrescentada a referência por rótulo, o mesmo modelo de fronteira passou a 86,8% de validade, com fidelidade estatisticamente indistinguível da geração direta de XML. Por outro caminho, o ajuste fino supervisionado levou um modelo de sete bilhões de parâmetros aos mesmos 86,8%, sem a gramática no contexto. Nenhuma das duas configurações alcança a validade da geração direta de XML.

A explicação organiza os achados do trabalho. O obstáculo não foi a novidade da notação, e sim a exigência de antecipação: um modelo autorregressivo gera da esquerda para a direita e não revisa o que já escreveu, de modo que uma notação que o obriga a marcar, ao emitir um nó, que um ramo futuro voltará a ele cria uma falha que nenhuma instrução remove. Removida essa exigência, a notação compacta torna-se emissível por instrução em contexto. O ajuste fino deixa, assim, de ser condição necessária da abordagem e passa a ser uma escolha de custo: ele dispensa a gramática no contexto e permite que um modelo pequeno substitua o de fronteira, ao preço de menor fidelidade.

6.1 CONTRIBUIÇÕES DO TRABALHO

Artefato técnico. Uma linguagem de domínio específico para descrição de processos, com gramática EBNF formalmente especificada, e um transpilador determinístico que a expande em XML BPMN 2.0. Sobre as 1.021 amostras ativas, a cadeia produz 100% de documentos

válidos contra o esquema oficial da OMG e preserva a equivalência topológica em 99,6% dos casos, com resíduo composto exclusivamente por arestas excedentes — isto é, o modo de falha remanescente nunca descarta lógica do processo de origem.

Evidência empírica sobre representações intermediárias. Três hipóteses foram refutadas com significância estatística e replicação em duas famílias de modelos: na primeira versão da notação, descrevê-la em *prompt* foi inferior a deixar o modelo emitir o formato que já domina, em fidelidade e em validade. A análise desse resultado negativo, registrada como emenda posterior à observação dos dados, mostrou que a inferioridade em fidelidade decorria quase inteiramente de falhas de forma, e desaparece com a correção da notação. O pré-registro do protocolo (Seção **??**) é o que permite relatar as duas coisas na ordem em que ocorreram.

Um princípio de desenho de notações para geração. Notações destinadas a serem emitidas por modelos autorregressivos não devem exigir que o gerador anuncie, ao escrever um elemento, que ele será referenciado adiante. A referência deve poder apontar para o que já está escrito, usando conteúdo presente no contexto do gerador, como o rótulo. No caso estudado, essa única alteração elevou a validade de um modelo de fronteira de 64,8% para 86,8%, ao custo de 1,7% de compressão. O princípio não depende da BPMN: aplica-se a qualquer notação com referências cruzadas, como identificadores em grafos, rótulos em linguagens de montagem ou âncoras em documentos.

Viabilidade econômica da especialização. A notação compacta reduz em 47% o custo por requisição do mesmo modelo de fronteira. O modelo ajustado emite a notação com a mesma validade desse modelo, sem a gramática no contexto, a um custo de duas a quatro vezes menor sob preço de mercado, com ou sem cache de *prompt*, e custou menos de um dólar para ser treinado; o patamar de validade foi replicado, em verificação exploratória, em duas famílias adicionais de modelos (Seção **??**). Sua fidelidade topológica é indistinguível da obtida pelo modelo de fronteira com a primeira versão da notação, porém inferior à obtida com a versão corrigida. A afirmação sustentada pelos dados não é de superioridade, e sim de uma troca explícita: mesma validade e menor fidelidade, a uma fração do custo.

Contribuição metodológica. Mediu-se o **teto humano** da métrica de fidelidade adotada (Seção **??**): duas referências de especialista para o mesmo processo concordam entre si em DF-F1 de apenas 0,1449, e em apenas 30,5% dos rótulos. Todos os métodos avaliados operam nessa vizinhança, e apenas um deles — a geração direta de XML pelo modelo de fronteira independente — a ultrapassa. Decorre daí que **métricas estruturais com identidade de nó por rótulo saturam próximo ao ruído** quando aplicadas a uma tarefa sem resposta única, e que

reportá-las contra um máximo teórico de 1,0 conduz a interpretação sistematicamente pessimista. Essa observação não depende da BPMN nem da linguagem aqui proposta, e é possivelmente a contribuição de maior alcance do trabalho.

6.2 LIMITAÇÕES

Validade externa do conjunto de treino. 95,3% das amostras de treinamento provêm de uma única fonte, o *handbook* corporativo da GitLab (Seção **??**). O modelo especializado é, portanto, treinado predominantemente sobre o estilo redacional de uma organização e avaliado contra um conjunto de domínio distinto. Ampliar o volume só reduz esse risco se as amostras vierem de fontes novas.

Replicação do ajuste fino apenas exploratória. O patamar de validade do modelo especializado foi replicado em duas famílias adicionais de modelos — Granite e MiMo, com 83,0% de validade em ambas contra 86,8% do braço principal (Seção **??**) —, de modo que a alegação não repousa sobre família única. As réplicas foram, contudo, executadas como braços exploratórios, fora do protocolo pré-registrado, e não carregam força confirmatória. O espaço de modelos tampouco se esgota nelas: o braço sobre uma quarta família foi invalidado por defeito de tokenização antes de qualquer medição, e o comportamento referencial divergiu entre as famílias — a taxa condicional de falha, estável entre 14% e 16% na família do modelo principal, sobe a 37–39% nas outras duas —, indício de que parte do resultado é propriedade do pré-treinamento, e não do método.

Teto imposto pelos dados de treinamento. A saída do próprio *pipeline* de geração de dados, pontuada contra as referências, alcança DF-F1 de 0,1533 (Seção **??**). Um modelo que aprendesse o conjunto de treinamento com fidelidade perfeita não superaria esse valor, o que limita superiormente qualquer ganho atribuível ao ajuste fino sob os dados atuais.

Limitações da métrica primária. Além da saturação já discutida, oito dos cinquenta e três itens de avaliação produzem DF-F1 nulo em **todos** os braços, incluindo os de geração direta de XML. A causa é convenção de nomenclatura: essas referências nomeiam atividades como orações em terceira pessoa com o ator por sujeito, ao passo que o protocolo instrui a forma imperativa. Como o casamento de rótulos não aplica radicalização morfológica, a similaridade fica estruturalmente abaixo do limiar. O efeito é uniforme entre braços e não altera ordenação alguma, mas deprime todos os valores absolutos em cerca de 18%.

Eixo de fluxos de mensagem não avaliável. Apenas dois dos cinquenta e três itens

possuem `messageFlow` na referência (Seção **??**), o que impede qualquer inferência sobre a capacidade de modelar comunicação entre participantes.

Dimensão do conjunto de avaliação. Com $n = 53$ itens e variância elevada, os contrastes têm potência estatística modesta. A ordenação entre braços chegou a inverter-se entre o conjunto completo e o subconjunto com referência múltipla, o que recomenda cautela na leitura de diferenças pequenas.

Correção da notação avaliada de forma exploratória e em um único modelo. A referência por rótulo foi introduzida depois de observados os resultados (emenda E-3), e seus contrastes não têm força confirmatória. Foi avaliada apenas com o modelo de fronteira independente; sua replicação no segundo modelo de fronteira, e no modelo especializado, que nunca observou essa forma em treino, não foi executada.

Custo do modelo especializado estimado, não observado. Não foi encontrado provedor que sirva adaptador LoRA próprio cobrando por *token*. O custo do modelo especializado é o preço de mercado de um modelo de mesmo porte, e, no cenário com cache de *prompt*, a proporção de desconto aplicada é hipótese. O tempo de inferência não foi comparado, pois os modelos foram executados em infraestruturas distintas.

6.3 TRABALHOS FUTUROS

Refinamento do casamento de rótulos. Radicalização morfológica e remoção de ator no início do rótulo endereçam diretamente a limitação mais severa identificada, e podem ser avaliadas como verificação de robustez sobre múltiplos limiares, preservando-se o valor congelado como número primário. Alterar o limiar após observar que a métrica produz valores baixos elevaria indistintamente todos os braços, motivo pelo qual não foi feito neste trabalho.

Referência por rótulo no modelo especializado. A forma de referência por rótulo, testada no modelo de fronteira (emenda E-3), não foi testada no modelo ajustado, cujo corpus contém apenas referências por identificador. É a extensão mais bem fundamentada que esta avaliação produziu: 15 das 21 falhas do modelo especializado são referências a identificadores não declarados, exatamente a classe que a forma por rótulo eliminou no modelo de fronteira. Reescrever o corpus nessa forma custa 2,31% em *tokens* e não exige dado novo, apenas retreino. Na mesma linha, cabe estender a forma por rótulo a nós ainda não escritos, que respondem por 7 das 21 falhas restantes do modelo de fronteira, e replicá-la no segundo modelo de fronteira.

Ampliação do corpus por fontes distintas. A curva de aprendizado executada sobre

frações aninhadas do conjunto (25%, 50% e 100%) mostrou que a perda de validação melhora monotonicamente com o volume, mas as métricas da tarefa não a acompanham — mais dados da mesma fonte compram verossimilhança, não capacidade (Seção **??**). A questão que permanece em aberto é o efeito de **fontes novas**, que combinam volume com diversidade estilística, sobre um treino hoje concentrado em 95,3% num único *handbook* (Seção **??**).

Aprendizado por reforço com recompensa verificável. A etapa prevista na Seção **??** permanece como extensão, com a ressalva ali registrada: a recompensa restrita a componentes verificáveis sem referência é explorável, pois premia a emissão da menor saída sintaticamente válida. Seu reparo exige um termo de ancoragem no texto de origem, cuja calibração constitui contribuição autônoma.

Decodificação restrita como linha de base concorrente. Garantir conformidade sintática durante a geração é alternativa direta ao que a correção da notação e o ajuste fino alcançaram aqui por outros caminhos, e o contraste entre as duas estratégias — uma que restringe o espaço de saída, outra que o internaliza — não foi avaliado.

Generalização para além da BPMN. A hipótese que motivou o trabalho é independente do domínio: aplica-se a qualquer esquema formal cuja serialização oficial seja verbosa. Avaliar o mesmo protocolo sobre outra linguagem de marcação testaria se os padrões observados — o efeito da forma de referência sobre a geração autorregressiva e o ganho de custo da especialização — são propriedades gerais da abordagem ou particularidades da BPMN.

REFERÊNCIAS

AA, H. van der *et al.* Challenges and opportunities of applying natural language processing in business process management. In: *Proceedings of the 27th International Conference on Computational Linguistics*. Santa Fe, New Mexico, USA: Association for Computational Linguistics, 2018. p. 2791–2801. Disponível em: <https://aclanthology.org/C18-1236/>.

Artificial Analysis. **Artificial Analysis Intelligence Index**. 2026. <https://artificialanalysis.ai/models>. Disponível em: <https://artificialanalysis.ai/models>. Acesso em: 24 out. 2026.

BELCAK, P.; HEINRICH, G.; DIAO, S.; FU, Y.; DONG, X.; MURALIDHARAN, S.; LIN, Y. C.; MOLCHANOV, P. Small language models are the future of agentic AI. **arXiv preprint arXiv:2506.02153**, 2025. Disponível em: <https://arxiv.org/abs/2506.02153>.

BELLAN, P.; AA, H. van der; DRAGONI, M.; GHIDINI, C.; PONZETTO, S. P. PET: An annotated dataset for process extraction from natural language text tasks. In: **Business Process Management Workshops: BPM 2022 International Workshops**. Cham: Springer, 2023. (Lecture Notes in Business Information Processing, v. 460), p. 315–321. Disponível em: <https://link.springer.com/chapter/10.1007/978-3-031-25383-6_23>.

BRISSARD, A.; CUPPENS, F.; ZOUAQ, A. **PMo Dataset**. Zenodo, 2025. Version 1.0.0. Disponível em: <https://zenodo.org/records/15857589>. Acesso em: 17 maio 2026.

BRISSARD, A.; CUPPENS, F.; ZOUAQ, A. What is the best process model representation? a comparative analysis for process modeling with large language models. In: **Proceedings of the AI4BPM Workshop at BPM 2025**. [S.l.: s.n.], 2025. Aceito para publicação; pré-print em arXiv:2507.11356.

BROWN, T. B.; MANN, B.; RYDER, N.; SUBBIAH, M. *et al.* Language models are few-shot learners. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [s.n.], 2020. v. 33, p. 1877–1901. Disponível em: <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html>.

CHEN, M.; TWOREK, J.; JUN, H.; YUAN, Q.; PINTO, H. Ponde de O. *et al.* Evaluating large language models trained on code. **arXiv preprint arXiv:2107.03374**, 2021. Disponível em: <https://arxiv.org/abs/2107.03374>.

COOTER, M. **Anthropic's AI services are too expensive, says Microsoft AI head**. 2026. CIO. Disponível em: <https://www.cio.com/article/4181777/anthropics-ai-services-are-too-expensive-says-microsoft-ai-head.html>. Acesso em: 6 set. 2026.

DETTMERS, T.; PAGNONI, A.; HOLTZMAN, A.; ZETTLEMOYER, L. QLoRA: Efficient finetuning of quantized LLMs. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [s.n.], 2023. v. 36. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract.html>.

DUMAS, M.; ROSA, M. L.; MENDLING, J.; REIJERS, H. A. **Fundamentals of Business Process Management**. 2. ed. Berlin, Heidelberg: Springer, 2018. Disponível em: <https://link.springer.com/book/10.1007/978-3-662-56509-4>.

Fireworks AI. **Deploying Trained Models**. 2026. <https://docs.fireworks.ai/fine-tuning/deploying-loras>. Disponível em: <https://docs.fireworks.ai/fine-tuning/deploying-loras>. Acesso em: 24 out. 2026.

FOWLER, M. **Domain-Specific Languages**. 1. ed. Boston: Addison-Wesley Professional, 2010. ISBN 978-0-321-71294-3. Disponível em: <https://www.informit.com/store/domain-specific-languages-9780321712943>.

FRIEDRICH, F.; MENDLING, J.; PUHLMANN, F. Process model generation from natural language text. In: **Advanced Information Systems Engineering**. Berlin, Heidelberg: Springer, 2011. (Lecture Notes in Computer Science, v. 6741), p. 482–496. Disponível em: <https://link.springer.com/chapter/10.1007/978-3-642-21640-4_36>.

GAO, X.; XIAO, B.; TAO, D.; LI, X. A survey of graph edit distance. **Pattern Analysis and Applications**, v. 13, n. 1, p. 113–129, 2010. Disponível em: <https://link.springer.com/article/10.1007/s10044-008-0141-y>.

Headroom Labs. **Headroom: Compress Tool Outputs, Logs, Files, and RAG Chunks Before They Reach the LLM**. 2026. <https://github.com/headroomlabs-ai/headroom>. Versão 0.27.0. Software de código aberto, licença Apache-2.0. Disponível em: <https://github.com/headroomlabs-ai/headroom>. Acesso em: 8 ago. 2026.

HU, E. J.; SHEN, Y.; WALLIS, P.; ALLEN-ZHU, Z.; LI, Y.; WANG, S.; WANG, L.; CHEN, W. LoRA: Low-rank adaptation of large language models. In: **International Conference on Learning Representations (ICLR)**. [s.n.], 2022. Disponível em: <https://openreview.net/forum?id=nZeVKeeFYf9>.

HUI, B.; YANG, J.; CUI, Z.; YANG, J.; LIU, D.; ZHANG, L.; LIU, T.; ZHANG, J.; YU, B.; LU, K. *et al.* Qwen2.5-Coder technical report. **arXiv preprint arXiv:2409.12186**, 2024. Disponível em: <https://arxiv.org/abs/2409.12186>.

JIANG, H.; WU, Q.; LIN, C.-Y.; YANG, Y.; QIU, L. LLMLingua: Compressing prompts for accelerated inference of large language models. In: **Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)**. Singapore: [s.n.], 2023. p. 13358–13376. Disponível em: <https://aclanthology.org/2023.emnlp-main.825/>.

KÖPKE, J.; SAFAN, A. Efficient LLM-based conversational process modeling. In: **Business Process Management Workshops (BPM 2024)**. Cham: Springer, 2025. (Lecture Notes in Business Information Processing, v. 534).

KOURANI, H.; BERTI, A.; SCHUSTER, D.; AALST, W. M. P. van der. ProMoAI: Process modeling with generative AI. In: **Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24), Demo Track**. International Joint Conferences on Artificial Intelligence Organization, 2024. p. 8708–8712. Disponível em: <https://www.ijcai.org/proceedings/2024/1014>.

MANGLER, J.; KLIEVTSOVA, N. **Textual Process Descriptions and Corresponding BPMN Models**. Zenodo, 2023. Version 1.0. Disponível em: <https://zenodo.org/records/7783492>. Acesso em: 17 maio 2026.

MERNIK, M.; HEERING, J.; SLOANE, A. M. When and how to develop domain-specific languages. **ACM Computing Surveys**, v. 37, n. 4, p. 316–344, 2005. Disponível em: <https://dl.acm.org/doi/10.1145/1118890.1118892>.

Object Management Group. **Business Process Model and Notation (BPMN), Version 2.0**. Needham, MA, USA, 2011. Documento OMG formal/2011-01-03. Disponível em: <https://www.omg.org/spec/BPMN/2.0/>.

OUYANG, L.; WU, J.; JIANG, X.; ALMEIDA, D.; WAINWRIGHT, C.; MISHKIN, P. *et al.* Training language models to follow instructions with human feedback. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [s.n.], 2022. v. 35. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract.html>.

POESIA, G.; POLOZOV, A.; LE, V.; TIWARI, A.; SOARES, G.; MEEK, C.; GULWANI, S. Synchromesh: Reliable code generation from pre-trained language models. In: **International Conference on Learning Representations (ICLR)**. [s.n.], 2022. Disponível em: <https://openreview.net/forum?id=KmtVD97J43e>.

ROPEK, L. **Uber caps employee AI spending after blowing through budget in four months**. 2026. TechCrunch. Disponível em: <https://techcrunch.com/2026/06/02/uber-caps-employee-ai-spending-after-blowing-through-budget-in-four-months/>. Acesso em: 6 set. 2026.

SANFELIU, A.; FU, K.-S. A distance measure between attributed relational graphs for pattern recognition. **IEEE Transactions on Systems, Man, and Cybernetics**, SMC-13, n. 3, p. 353–362, 1983.

SENNRICH, R.; HADDOW, B.; BIRCH, A. Neural machine translation of rare words with subword units. In: **Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)**. Berlin, Germany: Association for Computational Linguistics, 2016. p. 1715–1725. Disponível em: <https://aclanthology.org/P16-1162/>.

SHAO, Z.; WANG, P.; ZHU, Q.; XU, R.; SONG, J.; BI, X.; ZHANG, H.; ZHANG, M.; LI, Y. K.; WU, Y.; GUO, D. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. **arXiv preprint arXiv:2402.03300**, 2024. Disponível em: <https://arxiv.org/abs/2402.03300>.

SUGIYAMA, K.; TAGAWA, S.; TODA, M. Methods for visual understanding of hierarchical system structures. **IEEE Transactions on Systems, Man, and Cybernetics**, v. 11, n. 2, p. 109–125, 1981.

Together AI. **LoRA Inference**. 2026. <https://docs.together.ai/docs/lora-inference>. Disponível em: <https://docs.together.ai/docs/lora-inference>. Acesso em: 24 out. 2026.

Together AI. **Pricing**. 2026. <https://www.together.ai/pricing>. Disponível em: <https://www.together.ai/pricing>. Acesso em: 24 out. 2026.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [s.n.], 2017. v. 30. Disponível em: <https://papers.nips.cc/paper/7181-attention-is-all-you-need>.

VOELTER, M.; HADIAN, R.; KAMPIK, T.; BREITMAYER, M.; REICHERT, M. **Leveraging Generative AI for Extracting Process Models from Multimodal Documents**. 2024.

WESKE, M. **Business Process Management: Concepts, Languages, Architectures**. 3. ed. Berlin, Heidelberg: Springer, 2019. Disponível em: <https://link.springer.com/book/10.1007/978-3-662-59432-2>.

Apêndices

APÊNDICE A – Prompts dos braços experimentais

[A montar: regras comuns uma vez, depois a parte específica de cada braço, com o sha256 de cada prompt.]

APÊNDICE B – Prompts do juiz de fidelidade e da paráfrase

[A montar: rubrica completa, modelo da mensagem e prompt da paráfrase, com o *hash* da versão usada.]

APÊNDICE C – Piloto de escolha do modelo gerador

[A montar: tabela das três configurações e das três medidas.]

APÊNDICE D – Resultados das hipóteses originais

[A montar: H1 a H3 em DF-F1, como foram registradas.]